

# 基于粒子群算法的优化设计及烟幕干扰弹投放策略研究

## 摘要

烟幕干扰弹引爆后形成的烟幕云团具有遮蔽作用，是一种高效、低成本的干扰手段，在现代防空系统中具有重要应用。本文针对无人机投放烟幕干扰弹以干扰来袭导弹的问题，建立了基于运动学方程、时空匹配与智能优化的数学模型，求解干扰弹的投放策略，最大化烟幕对导弹的有效遮蔽时间。

**针对问题一**，建立**基于拟牛顿法的有效遮蔽判定模型**。首先根据给定初始条件建立运动学方程，求解坐标与时间的函数关系。然后通过 **L-BFGS-B 拟牛顿法** 判断真目标是否完全处于导弹和烟幕云团形成视锥内（**全遮挡**），即是否形成有效遮蔽。需要注意烟幕云团**不在导弹和真目标之间**的情况。求解得干扰弹对导弹的有效遮蔽时间为 **1.392s**。

**针对问题二**，建立最大化遮蔽时间的**单目标优化模型**，简化问题一的遮蔽判定模型。目标函数受多个连续决策变量约束，为非线性非凸函数，存在局部最优解，需要采用启发式算法。经过对比采用**粒子群算法**求解最大遮蔽时间。求解得干扰弹对导弹最大遮蔽时间为 **4.716s**，具体投放策略详见**表3**。

**针对问题三**，建立多干扰弹的**时序规划模型**。目标函数受投放时序约束。可先通过**贪心算法**，找到目标函数的一个可接受解，结合粒子群算法的快速收敛性和**禁忌搜索**的局部寻优能力求解。需要注意处理遮蔽区间重叠的时间段。求解得总有效遮蔽时间为 **6.682s**，具体投放策略详见**表6**和文件 **result1.xlsx**。

**针对问题四**，建立**多平台协同优化模型**。首先根据导弹的运行轨迹**划分空间**，对无人机进行**分类**，完成对无人机的任务分配，确定无人机的投放次序和投放空间，再通过基于**历史最优的边界收缩**粒子群算法求解。求解得总有效遮蔽时间 **12.544s**，具体投放策略详见**表7**和文件 **result2.xlsx**。

**针对问题五**，需要对**多目标优化**问题进行化简。首先通过贪心算法，遍历局部最优解同时实现了对决策变量的**降维**处理，并将问题转换为**线性规划**问题。求解得对 3 枚导弹的总有效遮蔽时间为 **17.413s**，具体投放策略详见**表10**和文件 **result3.xlsx**。

最后，我们对投放决策模型进行检验，验证其有较强的鲁棒性，同时综合评价了模型的优缺点。

**关键词：**烟幕干扰弹投放策略    有效遮挡判定    粒子群算法    贪心算法

# 一、问题重述

## 1.1 问题背景

在现代防空反导体系中，主动干扰手段成为应对导弹的重要补充。其中烟幕干扰弹通过形成烟幕云团实现对特定区域形成遮蔽。现采用“无人机挂载烟幕弹”的协同干扰模式，核心是通过无人机的飞行规划与干扰弹的时序控制构建有效遮蔽屏障。

## 1.2 题目信息

- 无人机投放烟幕干扰弹**间隔**  $t \geq 1\text{s}$ 。烟幕干扰弹投放后，仅受**重力作用**。
- 引爆干扰弹形成的球状云团以  $3\text{ m/s}$  **匀速下落**，在  $10\text{ m}$  范围提供 **20s 内**有效遮蔽。
- 来袭导弹飞行速度为  $300\text{ m/s}$ ，方向直指假目标。真目标为半径  $7\text{ m}$ 、高  $10\text{ m}$  的圆柱。
- 设假目标为原点，真目标下底面的圆心为  $(0, 200, 0)$ 。当警戒雷达监测到来袭导弹时，3 枚导弹和 5 架无人机的位置如下表所示。

|    |                     |     |                     |     |                      |
|----|---------------------|-----|---------------------|-----|----------------------|
| M1 | (20000, 0, 2000)    | FY1 | (17800, 0, 1800)    | FY4 | (11000, 2000, 1800)  |
| M2 | (19000, 600, 2100)  | FY2 | (12000, 1400, 1400) | FY5 | (13000, -2000, 1300) |
| M3 | (18000, -600, 1900) | FY3 | (6000, -3000, 700)  | -   | -                    |

## 1.3 问题提出

基于上述信息，需完成以下五个问题的模型建立和求解：

- **问题一**：利用 1 架无人机只投放 1 枚干扰弹干扰 1 枚导弹。设 FY1 以  $120\text{ m/s}$  飞向假目标，接到任务间隔  $1.5\text{ s}$  后投放干扰弹， $3.6\text{ s}$  后引爆。求干扰弹的**有效遮蔽时间**。
- **问题二**：利用 1 架无人机只投放 1 枚干扰弹干扰 1 枚导弹。确定一组 FY1 的飞行方向和速度、干扰弹投放点和起爆点，使得**遮蔽时间最长**。
- **问题三**：利用 **1 架**无人机**共投放 3 枚**干扰弹干扰 1 枚导弹，求干扰弹投放策略。
- **问题四**：利用 **3 架**无人机**各投放 1 枚**干扰弹干扰 1 枚导弹，求干扰弹投放策略。
- **问题五**：利用 **5 架**无人机**各至多投放 3 枚**干扰弹干扰 3 枚导弹，求干扰弹投放策略。

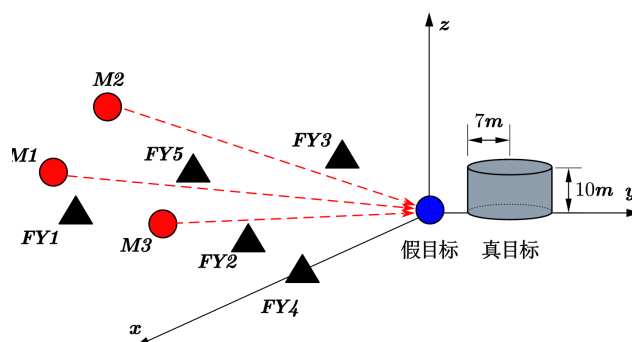


图 1 导弹来袭示意图

## 二、问题分析

### 2.1 问题一的分析

问题一中无人机飞行速度和方向等数据已知，构造**运动学方程**即可求解坐标。干扰弹投放后至引爆前运动为平抛运动，引爆后匀速下落。由此可得导弹和云团的三维坐标。导弹与真目标表面任意一点连线和云团球心间的距离应小于球的半径，即遮蔽有效。

### 2.2 问题二的分析

问题二为单个无人机投放单个干扰弹干扰单个导弹（简称“1-1-1”）。要求最大化遮蔽时长，并求出满足该最大化条件的投放策略。即求解一个**非线性单目标优化问题**。需要根据问题一建立的模型，进行遮蔽有效判定。此时目标函数受**4个决策变量**约束，需要简化问题一模型。且目标函数为非凸函数，可通过启发式算法如PSO等进行优化。

### 2.3 问题三的分析

问题三为单个无人机投放多个干扰弹干扰单个导弹（简称“1-3-1”）。需要规划三枚干扰弹依次投放的**时序**，实现遮蔽总时长的最大化。此外，还需考虑三个干扰弹可能存在**空间与时间重合**。此时目标函数受**8个决策变量**约束，相较问题二变量成倍增加，需要对问题二中的启发式算法改进。可结合**贪心算法**，只考虑当前干扰弹的最优解，先找到一个可接受解，再进行优化。

### 2.4 问题四的分析

问题四为多个无人机投放单个干扰弹干扰单个导弹（简称“3-1-1”）。不仅需要考虑三枚干扰弹的投放时序，还需要规划每架无人机的运行轨迹，和投放时间。此时目标函数受**12个决策变量**约束，是一个多平台协同优化问题。需要先对导弹轨迹进行空间划分，完成对不同无人机的任务分配，再进行分层优化。

### 2.5 问题五的分析

问题五为多个无人机投放多个干扰弹干扰多个导弹（简称“5-3-3”）。此时目标函数受**40个决策变量**约束，并且从此前的单目标优化问题变为了多目标优化问题。可对每架无人机和每枚导弹进行聚类 and 分配，将该问题简化为多个“1-3-1”或“3-1-1”问题，实现对决策变量的降维处理。同时也要考虑干扰弹同时对多个导弹实现有效遮挡的情况。

模型建立和求解的总体思路如图2所示。

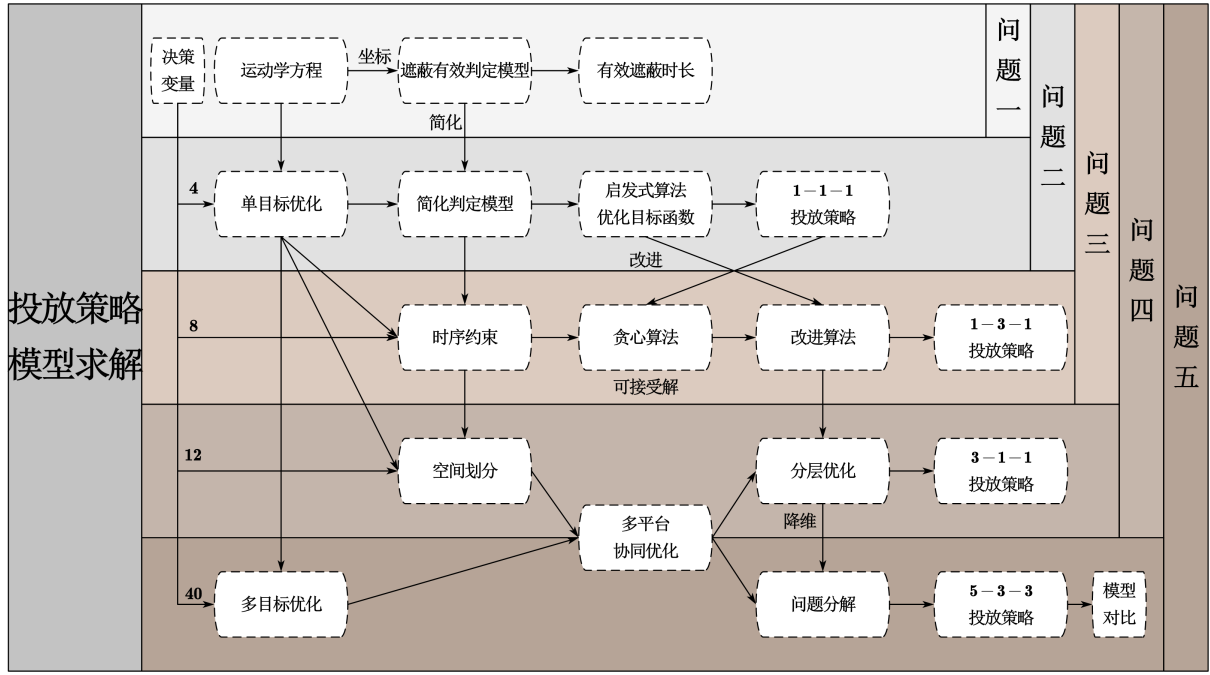


图 2 建模思路的总体框架

### 三、模型假设

- **假设 1:** 假设导弹、无人机、假目标均可视为质点。
- **假设 2:** 忽略空气阻力对导弹、烟幕干扰弹、烟幕云团、无人机的影响。
- **假设 3:** 忽略烟幕干扰弹爆炸形成云团、无人机改变飞行方向、无人机投放烟幕干扰弹所需时间间隔，均视为瞬时。且无人机改变方向为原地转向。
- **假设 4:** 导弹飞行、云团下沉、无人机等高度飞行均视作理想的匀速直线运动。
- **假设 5:** 假设重力加速度为国际公认的标准重力加速度值  $9.80665\text{m/s}^2$

### 四、符号说明

表 1 符号说明

| 符号                 | 说明                  | 量纲  |
|--------------------|---------------------|-----|
| $\theta_i$         | 编号为 $i$ 的无人机的水平飞行方向 | rad |
| $v_i$              | 编号为 $i$ 的无人机的水平飞行速度 | m/s |
| $(P_{drop})_i$     | 编号为 $i$ 的烟幕干扰弹的投放坐标 | m   |
| $(P_{blast})_i$    | 编号为 $i$ 的烟幕干扰弹的引爆坐标 | m   |
| $P_{Mi}$           | 编号为 $i$ 的导弹的坐标      | m   |
| $P_{Oi}$           | 编号为 $i$ 的烟幕云团球心的坐标  | m   |
| $T_{block}$        | 烟幕干扰弹的有效遮蔽时间        | s   |
| $\Delta t_{drop}$  | 无人机接收命令到投放的时间间隔     | s   |
| $\Delta t_{blast}$ | 无人机投放干扰弹到起爆的时间间隔    | s   |

## 五、问题一模型建立与求解

### 5.1 有效遮蔽判定模型建立

#### Step1: 确定导弹坐标 $P_M$

已知导弹 M1 初始坐标  $P_{M1}(0)$  和假目标坐标  $P_0$ ，可得导弹飞行的方向向量为：

$$\vec{n}_m = \frac{P_0 - P_{M1}}{\|P_0 - P_{M1}\|} \quad (1)$$

又知导弹飞行速度  $v_M = 300\text{m/s}$ ，得到  $t$  时刻导弹 M1 的坐标：

$$P_{M1}(t) = P_{M1}(0) + v_M \cdot t \cdot \vec{n}_m \quad (2)$$

#### Step2: 确定球形烟幕云团球心坐标 $P_O$

已知无人机 FY1 的初始坐标  $P_{fy1}(0)$ 、无人机的飞行速度  $v_f$ 、无人机的飞行方向  $\vec{n}_{fy}$  和接收命令到投放的时间间隔  $\Delta t_{drop}$ ，可得无人机的投放坐标：

$$P_{drop} = P_{fy1}(0) + v_f \cdot \Delta t_{drop} \cdot \vec{n}_{fy} \quad (3)$$

其中  $\vec{n}_{fy}$  为沿  $x$  轴负方向的单位向量。

烟幕干扰弹投放后受重力影响，同时由于惯性水平上继续向  $x$  轴负方向运动，作**平抛运动**。已知投放干扰弹到起爆的时间间隔  $\Delta t_{blast}$ ，可得干扰弹的起爆坐标：

$$P_{blast} = P_{drop} + v_f \cdot \Delta t_{blast} \cdot \vec{n}_{fy} - \frac{1}{2} \cdot g \cdot t_{blast}^2 \cdot \vec{e}_z \quad (4)$$

其中  $\vec{e}_z$  为沿  $z$  轴正方向的单位向量。

烟幕干扰弹起爆后形成的烟幕云团以  $v_0 = 3\text{m/s}$  的速度匀速竖直下落，得到  $t$  时刻烟幕云团的坐标：

$$P_O(t) = P_{blast} - v_0 \cdot (t - \Delta t_{drop} - \Delta t_{blast}) \cdot \vec{e}_z \quad (5)$$

$$t \geq \Delta t_{drop} + \Delta t_{blast}$$

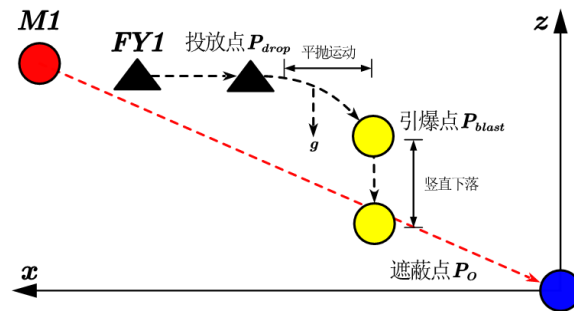


图 3 干扰弹投放和引爆示意图

### Step3: 建立视锥模型判定有效遮蔽

有效遮蔽定义为：过导弹坐标作过球形烟幕云团的切线形成一个**视锥**<sup>[2]</sup>，任取真目标圆柱表面的点，均包络在视锥内，即视为有效遮蔽。

已知真目标圆柱底面圆心  $P_C(0,0)$ 、圆柱半径  $R$  和圆柱高度  $h$  可将圆柱参数化为：

$$P_C(\theta_c, z) = P_C(0,0) + R \cdot \cos\theta_c \cdot \vec{e}_x + R \cdot \sin\theta_c \cdot \vec{e}_y + z \cdot \vec{e}_z \quad (6)$$

其中  $\theta_c \in (0, 2\pi), z \in (0, h)$ 。

在  $t$  时刻，任取圆柱上某一点  $P_C$ ，烟幕云团球心坐标到导弹坐标和该点坐标所确定直线的距离表达式为：

$$d_0 = \frac{\| (P_M - P_C) \times (P_M - P_O) \|}{\| P_M - P_C \|} \quad (7)$$

其中  $\times$  表示向量叉乘。

当  $d_0$  的最大值小于云团球心到视锥的距离，即云团半径，且云团位于导弹和圆柱之间时，即云团-导弹-圆柱三点形成锐角时，如图4-(a) 所示，能够保证真目标在视锥的阴影内。故  $t$  时刻，半径为  $r$  的烟幕干扰弹对真目标实现有效遮蔽的判定表达式为：

$$\max\{d_0\} \leq r \text{ \& } (P_M - P_C) \cdot (P_M - P_O) \geq 0 \quad (8)$$

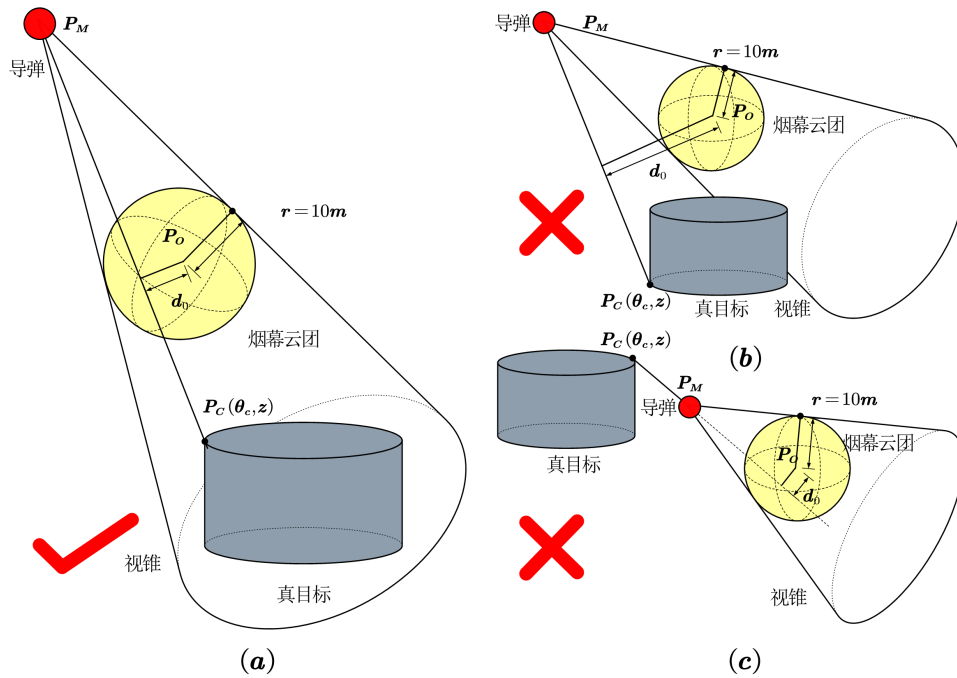


图 4 有效遮蔽判定示意图

云团在起爆  $\Delta t = 20s$  内可提供有效遮蔽，设引爆时间为  $t_d$ ，可得有效遮蔽时长为：

$$T_{block} = \int_{t_d}^{t_d + \Delta t} 1(\max d_0(t) \leq r) dt \quad (9)$$

#### Step4: 拟牛顿法求解 $d_0$ 最大值

综上所述,  $d_0$  最大值的求解可归纳为以下优化问题:

##### (1) 目标函数:

$$\max d_0(t, \theta_c, z) = \frac{\|(\mathbf{P}_M - \mathbf{P}_C) \times (\mathbf{P}_M - \mathbf{P}_O)\|}{\|\mathbf{P}_M - \mathbf{P}_C\|} \quad (10)$$

##### (2) 约束条件:

$$s.t. \begin{cases} \mathbf{P}_{M1}(t) = \mathbf{P}_{M1}(0) + v_M \cdot t \cdot \vec{n} \\ \mathbf{P}_{drop} = \mathbf{P}_{fy1}(0) + v_f \cdot \Delta t_{drop} \cdot \vec{n}_{fy} \\ \mathbf{P}_{blast} = \mathbf{P}_{drop} + v_f \cdot \Delta t_{blast} \cdot \vec{n}_{fy} - \frac{1}{2} \cdot t_{blast}^2 \cdot \vec{e}_z \\ \mathbf{P}_O(t) = \mathbf{P}_{blast} - v_0 \cdot (t - \Delta t_{drop} - \Delta t_{blast}) \cdot \vec{e}_z \\ \mathbf{P}_C(\theta_c, z) = \mathbf{P}_c(0, 0) + R \cdot \cos\theta_c \cdot \vec{e}_x + R \cdot \sin\theta_c \cdot \vec{e}_y + z \cdot \vec{e}_z \\ (\mathbf{P}_M - \mathbf{P}_C) \cdot (\mathbf{P}_M - \mathbf{P}_O) \geq 0 \\ \theta_c \in (0, 2\pi), z \in (0, h) \end{cases} \quad (11)$$

这是一个带边界约束的非线性优化问题, 所以我们采用的 **L-BFGS-B**<sup>[3]</sup> 拟牛顿法, 通过最小化  $d_0$  的负值实现该距离的最大化。L-BFGS-B 是对 L-BFGS 算法的改进, 核心在于为变量引入“边界约束”, 使算法能够高效地求解此类问题。其优化流程如图5所示。

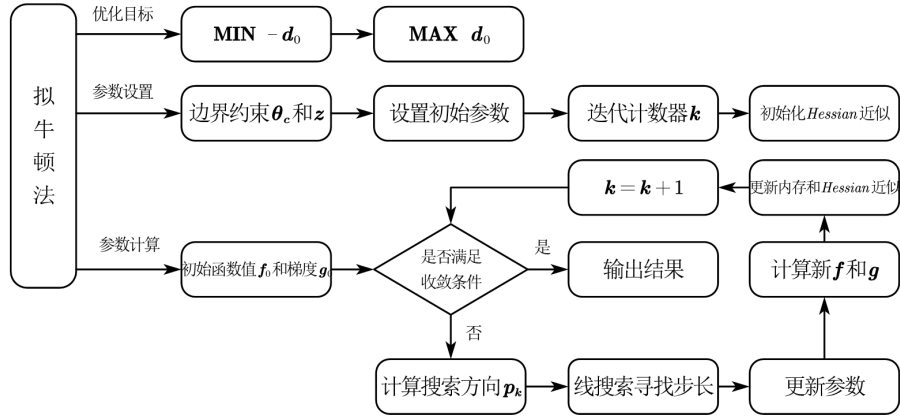


图 5 拟牛顿法最大化  $d_0$  流程图

## 5.2 问题一求解结果

表 2 问题一求解结果和分析

| 参数                        | 求解结果               |
|---------------------------|--------------------|
| 起爆坐标 $\mathbf{P}_{blast}$ | (17188,0,1736.496) |
| 遮蔽初始时间 $t_0$              | 8.056445s          |
| 遮蔽初始时间 $t_1$              | 9.448088s          |
| 有效遮蔽时间 $T_{block}$        | 1.391642s          |

## 六、问题二模型建立与求解

### 6.1 “1-1-1” 投放策略模型建立

#### Step1: $\max\{T_{block}\}$ 的单目标优化模型

问题二在问题一的基础上，引入了多个**连续型**决策变量。题目要求确定无人机 FY1 的飞行方向  $\theta_1$ 、速度  $v_1$ 、干扰弹投放坐标（或投放间隔  $\Delta t_{drop}$ ）、起爆坐标（或起爆间隔  $\Delta t_{blast}$ ），最大化遮蔽时间  $T_{block}$ ，即最佳投放策略。其与问题一运用相同的判定依据和坐标方程，仅是在决策层引入变量约束。

因此，我们需要借助问题一建立的有效遮蔽判定模型，给出一组使得遮蔽时间尽可能大的投放策略。根据题目要求，建立以下**单目标优化模型**：

(1) 目标函数：

$$\max \quad T_{block}(\theta_1, v_1, \Delta t_{drop}, \Delta t_{blast}) \quad (12)$$

(2) 约束条件：

$$s.t. \quad \begin{cases} 0 \leq \theta_1 \leq 2\pi \\ 70m/s \leq v_1 \leq 140m/s \\ 0 < \Delta t_{drop} < T_{max1} \\ 0 < \Delta t_{blast} < T_{max2} \end{cases} \quad (13)$$

其中：

- $T_{max1} = 13.75s$  为导弹 M1 飞至无人机 FY1 和真目标之间的最大时间。当  $t > T_{max1}$  时，飞机投放干扰弹无法实现遮蔽。

$$T_{max1} = \max\{t | \forall \theta_1 \in (0, 2\pi), (\mathbf{P}_M(t) - \mathbf{P}_C(t)) \cdot \mathbf{P}_M(t) - \mathbf{P}_O(t) \geq 0\} \quad (14)$$

- $T_{max2}$  为干扰弹引爆有效的最大时延。当无人机投放高度  $h \geq 1980m$  时，云团的遮蔽效果会失效；当  $h \leq 1980m$  时，云团因自由落体掉在底面无法实现遮挡。本题投放高度为 1800m，故  $T_{max2} = 19.17s$  为干扰弹投放后自由落体至地面的时间。

$$T_{max2} = \max\left\{\sqrt{\frac{2h_{fy1}}{g}}, 20\right\} \quad (15)$$

#### Step2: 遮蔽判定简化模型

由于本题以及第三问中的  $\mathbf{P}_{M1}(0)$ 、 $\mathbf{P}_{fy1}(0)$  和  $\mathbf{P}_C(0)$  满足：

$$\begin{aligned} \|\mathbf{P}_{M1}(0) - \mathbf{P}_C(0)\| &>> \|\mathbf{P}_{M1}(0) - \mathbf{P}_{fy1}(0)\| \\ \|\mathbf{P}_{M1}(0) - \mathbf{P}_C(0)\| &>> R, h \end{aligned} \quad (16)$$



故可近似将圆柱形真目标视为各若干个等距离质点，在判断是否形成有效遮挡时，等价于对质点遮挡的判断。并通过带入问题一模型检验简化模型合理性：当真目标视为标准圆柱时， $T_{block} = 1.391642s$ ；当视为质点时，为  $1.407534s$ ，误差  $\Delta e \approx 1\%$ 。

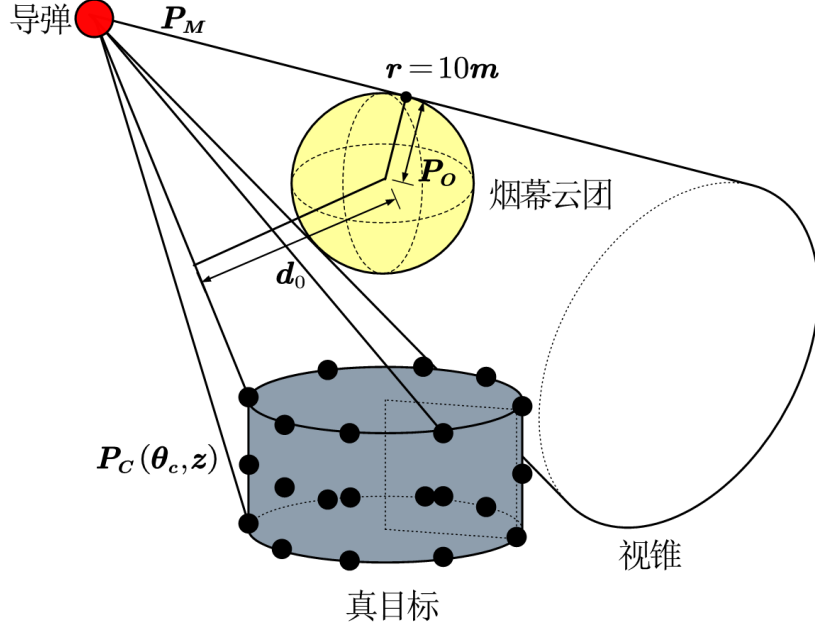


图 6 遮蔽判定简化示意图

## 6.2 “1-1-1” 投放策略模型的求解

综上所述，这是一个连续非线性非凸函数优化问题。对于这样一个多变量、不可微、可能存在多个局部最优解的优化问题，我们采用全局搜索能力强的粒子群算法（PSO）求解最优投放策略，得到出满足约束条件的最大遮蔽时间。优化步骤如下：

**Step1 初始化粒子群：**粒子数  $N = 80$ ，迭代次数  $T_{\max} = 160$ ，认知系数  $c_1 = 2.0$ ，社会系数  $c_2 = 2.0$ ，惯性权重  $w$  从  $0.90 \rightarrow 4$ ；随机初始化粒子位置  $x_i = (\theta, v_{uav}, t_{drop}, t_{explode})$ 。

**Step2 计算粒子适应度：**计算总有效遮蔽时长： $f(x_i) = -T(\theta, v_{fy}, t_{drop}, t_{blast})$ 。负号将最大化问题转换为最小化问题，便于 PSO 处理。

**Step3 更新个体与全局最优解：**对粒子的当前适应度  $f(x_i)$  与其历史最优适应度  $p_{best}(x_i)$  比较，若  $f(x_i) < p_{best}(x_i)$ ，更新  $p_{best}(x_i) = f(x_i)$  并记录位置；再比较所有粒子的  $p_{best}(x_i)$ ，取最小值作当前全局最优解  $g_{best}$ ，对应最优投放策略。

**Step4 更新粒子速度与位置：**根据 PSO 动力学方程更新每个粒子的速度和位置：

**Step5 边界条件：**对粒子的位置进行约束，确保决策变量的物理可行性，同并限制速度变化范围，避免振荡。

**Step6 终止条件：**若达到最大迭代次数  $T_{\max} = 160$ ，则输出全局最优解  $g_{best}$ ，解码得到最优投放策略： $(\theta^*, v_{uav}^*, t_{drop}^*, t_{blast}^*)$ 。并计算对应的最大有效遮蔽时长  $T_{block} = -g_{best}$ ；反之，返回 Step2 继续迭代优化。

6.3 问题二求解结果和分析

解得投放策略和最大遮蔽时间如下表所示:

表 3 问题二无人机干扰弹投放策略

| 参数                   | 求解结果                      |
|----------------------|---------------------------|
| FY1 飞行方向 $\theta_1$  | 0.1023rad or 5.86°        |
| FY1 飞行速度 $v_1$       | 116.27m/s                 |
| 烟幕干扰弹投放点 $P_{drop}$  | (17857.8300,5.93,1800.00) |
| 烟幕干扰弹起爆点 $P_{blast}$ | (17915.661,11.87,1798.78) |
| 有效遮蔽时间 $T_{block}$   | 4.716s                    |

1.PSO 的收敛性分析:

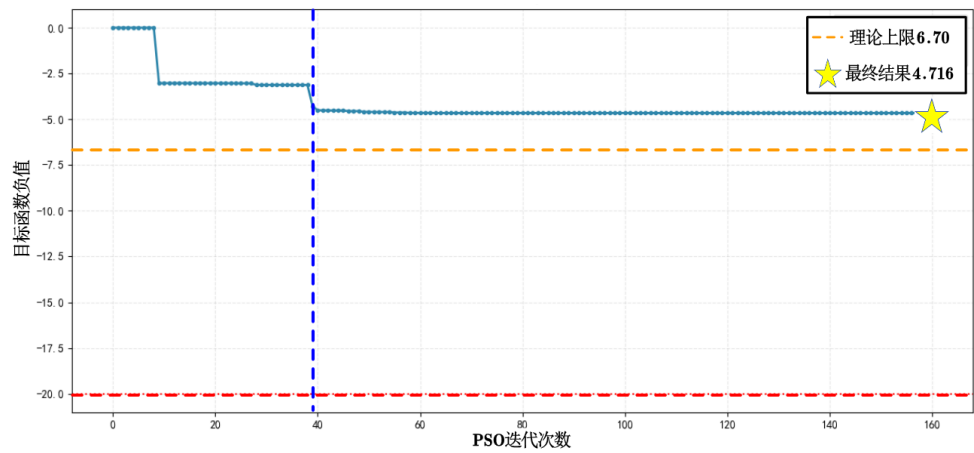


图 7 问题二粒子群优化收敛曲线

从图7中看出,该粒子群算法具有较快的收敛速度,仅用 40 次左右迭代就找到最优解。图7中的理论上限将在模型检验部分给出解释。

2. 遮蔽判定简化模型误差分析:

采用不同的遮蔽判定模型得到的不同最大遮蔽时间如下表所示:

表 4 不同判定模型下的结果

| 模型          | 拟牛顿法   | 20 质点  | 40 质点  | 82 质点  |
|-------------|--------|--------|--------|--------|
| 迭代次数        | 60+    | 100+   | 80+    | 40     |
| 运行时间        | 30+min | 3min   | 5min   | 10min  |
| $T_{block}$ | 4.614s | 4.922s | 4.789s | 4.716s |

从结果看出,82 质点简化判定模型求解结果近似拟牛顿法,且具有更快的收敛速度和较低的时间复杂度,误差  $\Delta e \approx 2.21\%$ 。

## 七、问题三模型建立与求解

### 7.1 建模思路

问题三在问题二的基础上，**投放干扰弹数量增加到 3 枚**。需要建立模型决策三枚干扰弹投放和引爆的时序，即  $t_{dropi}$  和  $t_{blasti}$ 。且无人机相邻两次投放间隔  $\geq 1$ 。由于无人机方向确定后无法调整，故投放坐标和引爆坐标受 FY1 无人机的速度、方向约束。受 8 个决策变量约束，目标函数为总遮蔽时长，每枚干扰弹的遮蔽区间之间可不连续，若连续则需要去除遮蔽时长中的重叠部分。本题的时序结构如图11所示。

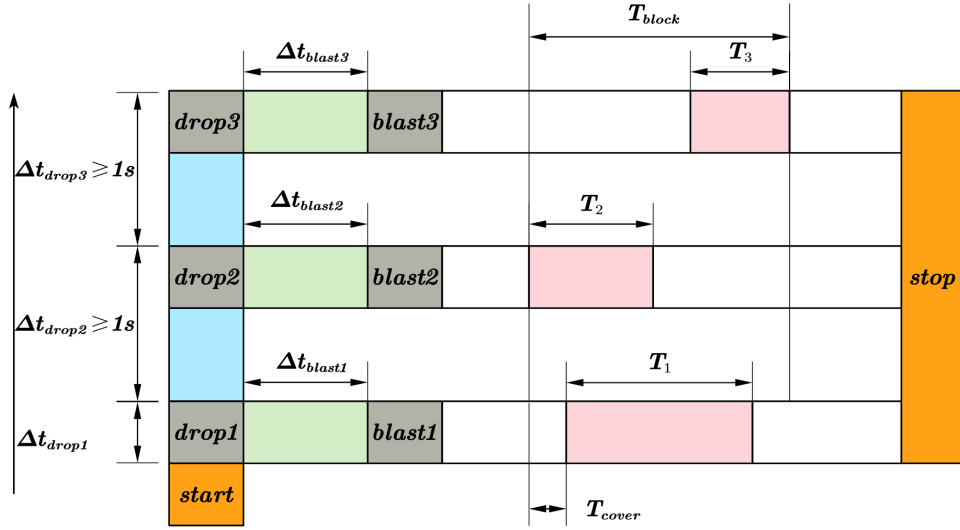


图 8 问题三有效遮蔽时序图

### 7.2 “1-3-1” 投放策略模型建立

#### Step1: 重叠区间处理模型

计算总遮蔽时长需要对遮蔽区间的重叠部分进行处理，设每个干扰弹遮蔽区间分别为  $T_1, T_2, T_3$ ，总遮蔽时长为：

$$T_{block} = T_1 + T_2 + T_3 - T_{cover} \quad (17)$$

其中  $T_{cover}$  为总重叠时间。

根据容斥原理，总遮蔽时间为：

$$T_{block} = |T_1 \cup T_2 \cup T_3| = |T_1| + |T_2| + |T_3| - |T_1 \cap T_2| - |T_3 \cap T_2| - |T_3 \cap T_1| + |T_1 \cap T_2 \cap T_3| \quad (18)$$

#### Step2: 时序约束下的 $\max\{T_{block}\}$ 的单目标优化模型

根据上述分析，建立以下时序决策模型：

##### (1) 目标函数：

$$\max_{i=1,2,3} \sum_{i=1}^3 T_{block}(\theta_1, v_1, (\Delta t_{drop})_i, (\Delta t_{blast})_i) - T_{cover} \quad (19)$$

(2) 约束条件:

$$s.t. \begin{cases} 0 \leq \theta_1 \leq 2\pi \\ 70m/s \leq v_1 \leq 140m/s \\ 0 < \sum_{i=1}^3 (\Delta t_{drop})_i < T_{max1} \\ 0 < (\Delta t_{blast})_i < T_{max2} \\ |(\Delta t_{drop})_i - (\Delta t_{drop})_j| \geq 1s, i \neq j \end{cases} \quad (20)$$

### 7.3 “1-3-1” 投放策略模型求解

#### Step1: 采用自适应步长

首先对问题二中的粒子群优化算法进行改进,将原本的0.01固定步长改为(0.001,0.4)范围的自适应步长,根据导弹与目标的**实时距离**动态调整时间步长:

$$step = step_{min} + (step_{max} - step_{min}) \cdot \frac{d}{d_{max}} \quad (21)$$

- 距离远时,使用较大步长(最大0.400s),提高计算**效率**;
- 距离近时:使用较小步长(最小0.001s),提高计算**精度**。

经过优化后,问题二求解最大遮蔽时间的计算速度大大提高(30min到10min),同时计算精度提高1位。

#### Step2: 采用贪心算法寻找可接受解

基于问题二结果,只考虑当前干扰弹最佳投放策略,求得一个可接受解  $T_{block} = 5.625s$ 。

#### Step3: 结合禁忌算法

由于本题的决策变量增加,存在大量局部最优解,单一地使用粒子群算法容易“早熟收敛”,即过早地陷入一个局部最优点,尤其在这种高维离散问题上。后期粒子群收敛速度慢,且缺乏高效的跳出局部最优的机制。故本题采取**改进型粒子群算法(PSO+TS)**。

表5 PSO和TS优缺点分析

| 算法 | PSO                                | TS                              |
|----|------------------------------------|---------------------------------|
| 优点 | 1. 全局探索能力强<br>2. 前期快速向可能区域收敛       | 1. 局部探索能力强<br>2. 能有效避免循环并跳出局部最优 |
| 缺点 | 1. 容易过早陷入局部最优点<br>2. 缺乏有效的机制跳出局部最优 | 搜索策略单一                          |

对于问题三这种多决策变量且需要协同优化以达成单一目标的问题，TS+PSO 的混合策略<sup>[4]</sup>能够极大地改进优化效果。

## 7.4 问题三求解结果和分析

表 6 问题三无人机干扰弹投放策略

| FY1 方向 | 0.1807rad or 10.35° |                      |       |               |
|--------|---------------------|----------------------|-------|---------------|
| FY2 速度 | 101.80m/s           |                      |       |               |
| 干扰弹编号  | 投放坐标                | 起爆坐标                 | 干扰时长  | 有效区间          |
| 1      | (17800,0,1800)      | (17813,2.455,1799.9) | 3.285 | [4.397,7.682] |
| 2      | (17900,18.29,1800)  | (17900,18.29,1800)   | 3.463 | [1.0,4.463]   |
| 3      | *                   | *                    | *     | *             |

注：\* 处表示无法进行有效遮挡，同下。

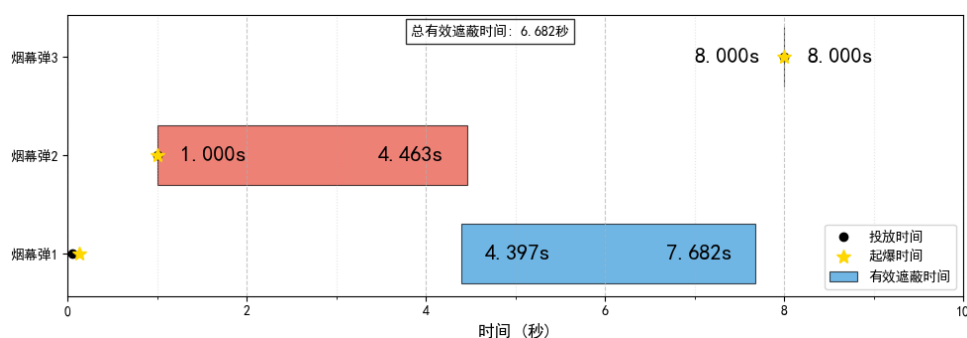


图 9 问题三干扰弹有效遮蔽时序图

从图10可知，采用自适应步长并结合禁忌算法后的粒子群算法在决策变量相较问题二翻倍的情况下，仅采用 25 次左右的迭代次数完成了快速收敛（问题二使用了 40 次迭代完成收敛）。

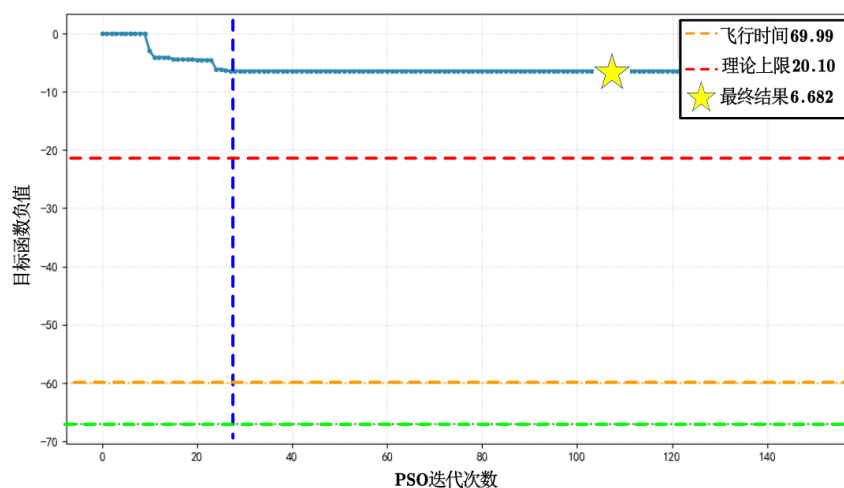


图 10 问题三粒子群优化收敛曲线

## 八、问题四模型建立与求解

### 8.1 建模思路

问题四在问题二的基础上，无人机数量增加到 3 架，各自投一枚干扰弹。此时干扰弹投放的时间间隔不受时序约束，且每架无人机的速度、方向不同，故运行的轨迹也不同。不仅要求解三枚干扰弹投放和引爆的时序，同时要规划不同无人机的轨迹，这是一个多平台协同优化问题，受 12 个决策变量约束，需要进行空间划分简化模型。本题的时序结构如图11所示。

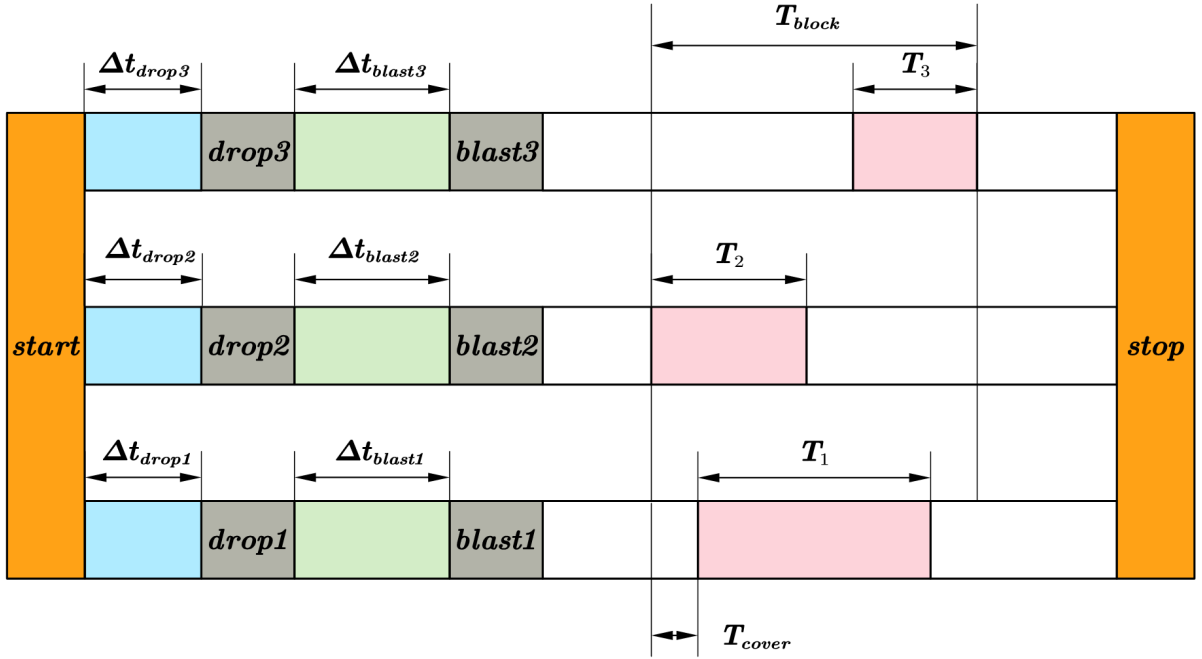


图 11 问题四有效遮蔽时序图

### 8.2 “3-1-1” 投放策略模型建立

**Step1: 协同优化模型** 根据上述分析，建立以下模型：

(1) 目标函数：

$$\max_{i=1,2,3} \sum_{i=1}^3 T_{block}(\theta_i, v_i, (\Delta t_{drop})_i, (\Delta t_{blast})_i) \quad (22)$$

(2) 约束条件：

$$s.t. \begin{cases} 0 \leq \theta_1, \theta_2, \theta_3 \leq 2\pi \\ 70m/s \leq v_1, v_2, v_3 \leq 140m/s \\ 0 < (\Delta t_{drop})_i < (T_{max1})_i \\ 0 < (\Delta t_{blast})_i < (T_{max2})_i \end{cases} \quad (23)$$

**Step2: 空间划分**

为了简化模型，需要根据导弹运行轨迹与真假目标、三架无人机的相对位置进行初步的**空间划分**，以便分配投放任务。

首先求解导弹  $P_M$ 、真目标  $P_O$ 、假目标  $P_C$  三点确定的平面。利用向量的叉积求平面的法向量：

$$\vec{n} = \overrightarrow{MO} \times \overrightarrow{MC} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ \overrightarrow{MO} \cdot \vec{e}_x & \overrightarrow{MO} \cdot \vec{e}_y & \overrightarrow{MO} \cdot \vec{e}_z \\ \overrightarrow{MC} \cdot \vec{e}_x & \overrightarrow{MC} \cdot \vec{e}_y & \overrightarrow{MC} \cdot \vec{e}_z \end{vmatrix} = (A, B, C) \quad (24)$$

其中  $\overrightarrow{MO} = P_M - P_O$ ， $\overrightarrow{MC} = P_M - P_C$ 。

将参数  $(A, B, C)$  和三点中任意一点坐标代入平面方程公式即可确定平面。

$$Ax + By + Cz + D = 0 \quad (25)$$

再根据**垂线参数方程**，求得无人机在该平面的投影坐标  $(x, y, z)$  和无人机到平面的垂直距离：

$$\begin{aligned} x &= x_0 - A \cdot \frac{Ax_0 + By_0 + Cz_0 + D}{A^2 + B^2 + C^2}, \\ y &= y_0 - B \cdot \frac{Ax_0 + By_0 + Cz_0 + D}{A^2 + B^2 + C^2}, \\ z &= z_0 - C \cdot \frac{Ax_0 + By_0 + Cz_0 + D}{A^2 + B^2 + C^2}. \end{aligned} \quad (26)$$

$$d_0 = \frac{Ax_0 + By_0 + Cz_0 + D}{\sqrt{A^2 + B^2 + C^2}} \quad (27)$$

根据无人机落在平面上的投影和无人机到平面的距离权重划分空间，进行初步的任务分配，即各无人机负责被分配区域的干扰弹投放。

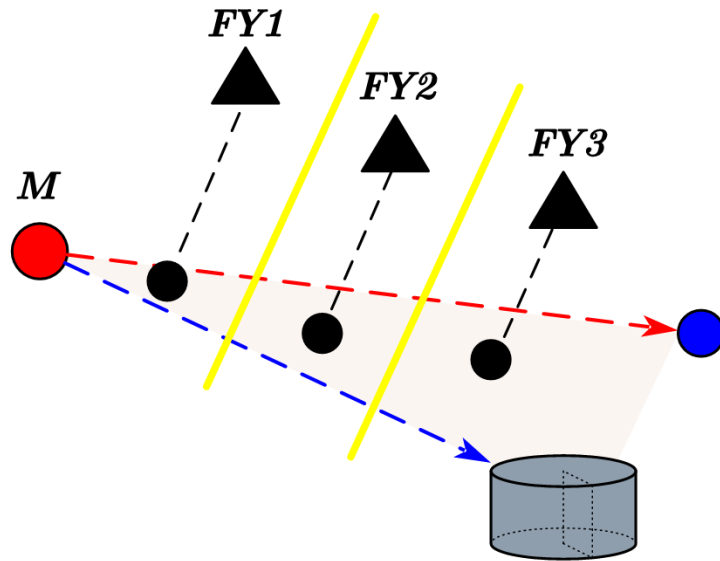


图 12 空间划分示意图

8.3 “3-1-1” 投放策略模型求解

Step1: 贪心算法求可接受解

为了寻找一个局部最优解，可将问题四的“3-1-1”模型拆解为3个问题三中的“1-1-1”模型，分别求解各架无人机对M1的最大遮蔽时间，处理重叠区域后，得到一个可接受解10.8s。

Step2: 基于“边界收缩”改进的粒子群算法

本题使用粒子群算法进行全局搜索计算量巨大，易受困于局部最优解而无法收敛到全局最优解。不妨考虑缩小搜索空间，通过概率分布的半定量判断得到决策变量解最有可能出现的小范围。将该问中每一架无人机参数代入问题二，求出在仅有一个无人机干扰下的最佳参数，以该参数附近设计一个合理的小区间进行搜索，即最优策略是使每一架飞机尽量多产生有效遮蔽。

8.4 问题四求解结果

表 7 问题四无人机干扰弹投放策略

| 无人机编号   | FY1            | FY2              | FY3              |
|---------|----------------|------------------|------------------|
| 无人机运动方向 | 5.86           | 256.61           | 114.91           |
| 无人机运动速度 | 116.27         | 94.48            | 129.89           |
| 干扰弹投放坐标 | 17857          | 11824.2          | 4915.505         |
|         | 5.9354717      | 661.49           | -664.723         |
|         | 1800           | 1400             | 700              |
| 干扰弹起爆坐标 | 17915.6624     | 11676.533        | 4566.845         |
|         | 11.870         | 41.178           | 86.0564          |
|         | 1798.775       | 1176.81          | 500.9858         |
| 干扰区间    | [0.987, 5.601] | [15.750, 19.766] | [37.957, 41.872] |
| 干扰时间    | 4.614          | 4.015            | 2.914            |

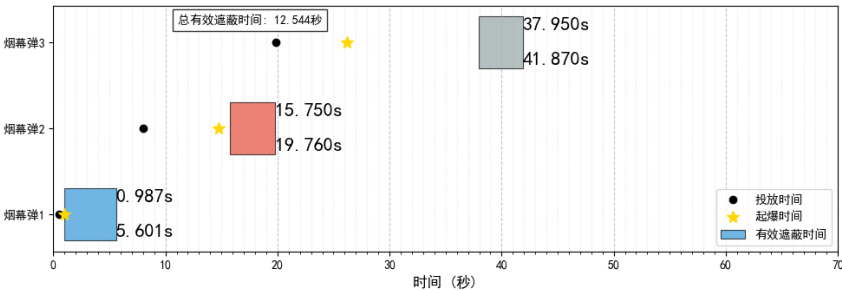


图 13 问题四干扰弹有效遮蔽时序图



## 九、问题五模型建立与求解

### 9.1 建模思路

问题五在前四问的基础上，无人机数量增加到 5 架，投放干扰弹数量增加到 15 枚干扰弹，导弹数量增加到 3 枚，从单目标优化问题变为**多目标优化问题**。为了简化优化模型，可忽略一枚干扰弹可有效遮蔽多枚导弹的情况。故对问题四中的划分模型进行改进，将 5 架无人机分配给 3 枚导弹，即将“5-3-3”的多目标优化问题转化为若干个单目标优化问题。

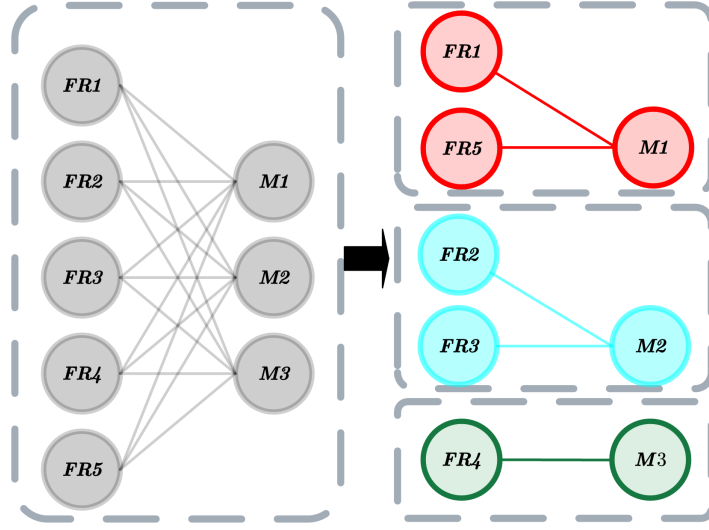


图 14 简化多目标优化问题示意图

### 9.2 “5-3-3” 投放策略模型建立

### 9.3 多目标优化模型建立

本题的决策目标是有以下三种解释：

- 最大化对各枚导弹的遮蔽时间的**总和**。
- 最大化对各枚导弹的遮蔽时间的**加权总和**，加权系数与距离相关。
- 最大化对各枚导弹**同时处于有效遮蔽状态**时间的最大值。

根据上述分析，建立如下多目标优化模型：

#### (1) 目标函数：

$$\max_{i,j} \begin{cases} \sum_{j=1}^3 \sum_{i=1}^5 T_{block}(\theta_i, v_i, (\Delta t_{drop})_{ij}, (\Delta t_{blast})_{ij}) \\ \sum_{j=1}^3 w_j \cdot \sum_{i=1}^5 T_{block}(\theta_i, v_i, (\Delta t_{drop})_{ij}, (\Delta t_{blast})_{ij}) \\ \bigcup_{i=1}^n T_{block_i}(\theta_i, v_i, (\Delta t_{drop})_{ij}, (\Delta t_{blast})_{ij}) \end{cases} \quad (28)$$

(2) 约束条件:

$$s.t. \begin{cases} 0 \leq \theta_i \leq 2\pi, i = 1, 2, 3, 4, 5 \\ 70m/s \leq v_i \leq 140m/s \\ 0 < (\Delta t_{drop})_{ij} < (Tmax1)_i, j = 1, 2, 3 \\ 0 < (\Delta t_{blast})_{ij} < (Tmax2)_i \end{cases} \quad (29)$$

根据图15-(a)所示，当真目标处于两个视锥的重叠部分时，烟幕云团才可能同时遮蔽两个导弹。由于导弹为质点且距离真目标较远，可将其简化为15-(b)的平面示意图。将云团看作半径 10 的圆形，真目标看作半径 7 米的圆形，仅当云团和真目标距离  $d_0$  足够小时才能满足真目标同时处于两枚导弹的阴影中。经过试验发现：由于 3 枚导弹间隔距离较大，云团相对较小，距离真目标较远，几乎不存在某一时刻同时满足三枚导弹处于遮蔽状态。故选择第一种目标函数。

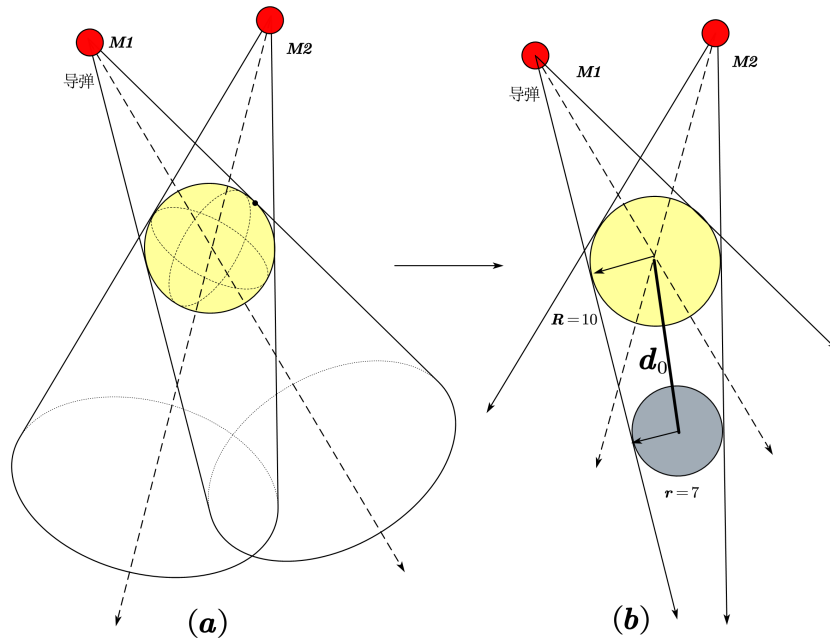


图 15 多导弹遮蔽示意图

#### 9.4 “5-3-3” 投放策略模型求解

##### Step1: 化简为“1-3-1”模型降维

问题五决策变量剧增，直接求解的复杂度巨大，故为了寻找一个局部最优解，可在不考虑单个干扰弹可遮蔽多个导弹的情况。将问题五的“5-3-1”模型拆解为 15 个问题三中的“1-3-1”模型，通过调用 15 次问题三的 python 代码分别求解各架无人机对 M1、M2、M3 的最大遮蔽时间：

##### Step2: 贪心算法求局部最优解

如表8所示，FY1-M2、F1-M3、FY4-M1、FY4-M2、FY4-M3、FY5-M2 这 6 中可能

表 8 3\*5 对 “1-3-1” 模型求解结果

| 遮蔽时间 | FY1   | FY2   | FY3   | FY4 | FY5   |
|------|-------|-------|-------|-----|-------|
| M1   | 6.682 | 3.968 | 2.677 | 0   | 3.931 |
| M2   | 0     | 4.057 | 2.741 | 0   | 0     |
| M3   | 0     | 3.895 | 2.733 | 0   | 3.508 |

性均不存在故只需要考虑另外 9 种情况。9 种投放策略对单一导弹的遮蔽区间为：

表 9 3\*5 对 “1-3-1” 模型求解结果

| 策略     | 遮蔽区间            |
|--------|-----------------|
| FY1-M1 | [1.0,7.682]     |
| FY2-M1 | [16.877,20.844] |
| FY2-M2 | [12.143,16.2]   |
| FY2-M3 | [15.865,19.76]  |
| FY3-M1 | [35.941,38.617] |
| FY3-M2 | [26.135,28.876] |
| FY3-M3 | [31.676,34.409] |
| FY5-M1 | [18.162,22.094] |
| FY5-M3 | [18.061,21.569] |

根据表8和表9的数据，通过绘制导弹与无人机遮蔽区间的时间序列，我们能够更直观地做出决策：

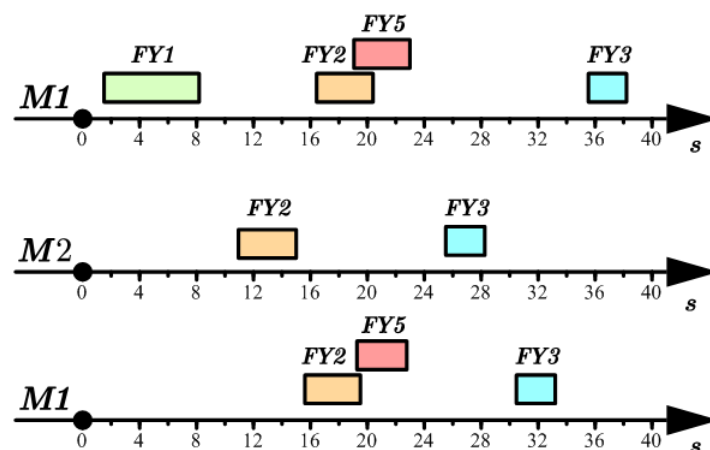


图 16 问题五干扰弹有效遮蔽时序图

此时，问题五从一个多目标规划问题，被简化为一个线性规划模型，且能通过简单的逻辑推理，直接作出此刻的最佳决策：

**Step1:** FY1 能且仅能对 M1 进行干扰，其遮蔽区间与其他无人机遮蔽区间均不重叠。  
**Step2:** FY2-M123 三者中 FY2-M2 最大，且是唯一不与其他遮蔽区间的方案。  
**Step3:** FY2 选择后，FY5-M13 均不与其他遮蔽区间重叠，选择较大的 FY-5-M1  
**Step4:** 选择最大的 FY3-M2。

解得对导弹 M1、M2、M3 的遮蔽时间分别为 10.613s、6.798s、0s。总遮蔽时长为 17.411s，此为本文的一个局部最优解，未考虑同一枚干扰弹可遮蔽多枚导弹等情况。

### 9.5 问题五求解结果和分析

表 10 问题五无人机干扰弹投放策略

| 干扰弹 | 无人机 | 速度     | 方向     | 投放坐标                          | 引爆坐标                          | 遮蔽时间  | 干扰导弹 |
|-----|-----|--------|--------|-------------------------------|-------------------------------|-------|------|
| 1   | FY1 | 101.80 | 10.35  | (17800<br>0<br>1800)          | (17813<br>2.45<br>1799.9)     | 3.285 | M1   |
| 2   | FY1 | 101.80 | 10.35  | (17900<br>18.29<br>1800)      | (17900<br>18.29<br>1800)      | 3.463 | M1   |
| 3   | FY2 | 106.23 | 290.77 | (12297<br>616.3<br>1400)      | (12370<br>424.8<br>1381.8)    | 4.057 | M2   |
| 4   | FY3 | 132.44 | 103.8  | (5352.2<br>-354.96<br>700)    | (5199.3<br>269.55<br>584.51)  | 2.741 | M2   |
| 5   | FY5 | 120.66 | 99.70  | (12700.8<br>-249.9<br>1300.0) | (12654.8<br>19.198<br>1274.9) | 3.931 | M1   |

表 11 模型对比

| 模型    | 1-1-1 | 1-3-1 | 3-1-1  | 5-3-3  |
|-------|-------|-------|--------|--------|
| 总遮蔽时间 | 4.716 | 6.682 | 12.544 | 17.413 |
| 对比提升度 | -     | 41.8% | 166%   | 269%   |

## 十、模型检验

### 10.1 有效遮蔽时间模型检验

为了检验烟幕云团对真目标的最大有效遮蔽时间计算结果是否合理，故计算烟幕云团对对导弹遮蔽时间的**上限值**。如图17-(a)所示，我们假设导弹、真目标、假目标均为质点，坐标分别为  $P_M$ 、 $P_C$ 、 $P_0$ ，则三者构成夹角为：

$$\alpha = \arccos \frac{(P_M - P_0) \cdot (P_M - P_C)}{\| (P_M - P_0) \| \cdot \| (P_M - P_C) \|} \quad (30)$$

当导弹距离较远时有  $\alpha \approx 0$ ，不妨假设真目标和假目标坐标重合。此时遮蔽时间的上限值计算如图17-(b)所示。在理想情况下，云团在导弹运行轨迹的正上方下落，下落过程中云团与导弹运行轨迹相切的两个时间点的间隔即为遮挡范围，期间下落高度为：

$$d_{max} = 2r / \frac{\| \vec{v}_M \|}{\| \vec{v}_M \cdot \vec{e}_x + \vec{v}_M \cdot \vec{e}_y \|} \quad (31)$$

计算得烟幕云团对对导弹遮蔽时间的**上限值**为：

$$(T_{block})_{max} = d_{max} / v_0 \quad (32)$$

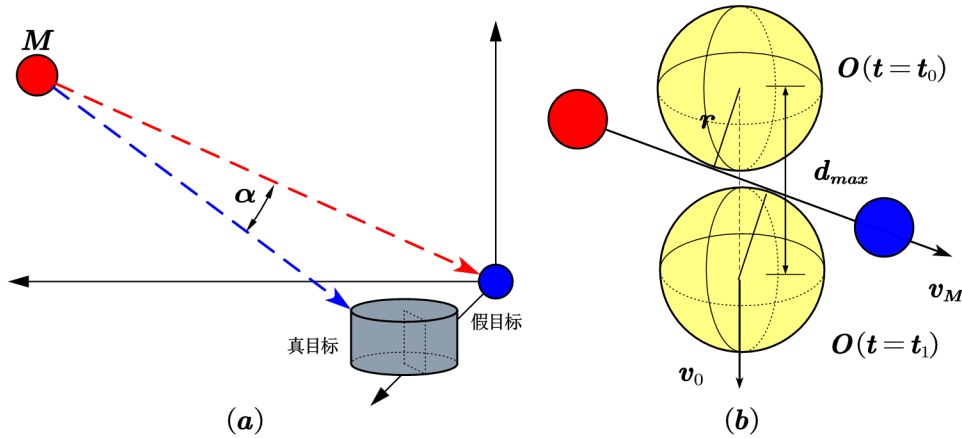


图 17 遮蔽时间上限计算示意图

通过以上模型得到各个导弹的遮蔽理论上限值为：

表 12 理想情况下遮蔽时间上限值

| 导弹编号 | $(T_{block})_{max}$ | 是否符合模型 |
|------|---------------------|--------|
| M1   | 6.699914            | 是      |
| M2   | 6.707245            | 是      |
| M3   | 6.703631            | 是      |

对比各问题的求解结果，单个干扰弹  $T_{block}$  均小于理论上限，验证了模型的正确性。

## 10.2 有效遮蔽判定模型误差分析

我们将通过简化模型得到的最佳投放策略带入问题一模型中进行检验，得到各个问题在精确模型和简化模型下的相对误差，如表13所示。

表 13 精确模型和简化模型相对误差

| 题号   | 问题一    | 问题二    | 问题三    | 问题四     | 问题五    |
|------|--------|--------|--------|---------|--------|
| 决策变量 | 0      | 4      | 8      | 12      | 40     |
| 量化点数 | 1      | 82     | 40     | 191     | 1      |
| 拟牛顿法 | 1.392s | 4.716s | 6.356s | 11.76s  | 16.062 |
| 化简模型 | 1.408s | 4.614s | 6.682s | 12.544s | 17.413 |
| 相对误差 | 1.15%  | 2.21%  | 4.87%  | 6.66%   | 8.4%   |

上述数据表面在使用将圆柱形真目标视为若干质点的简化模型的情况下，也能够有较为精确的求解结果；且随着问题决策变量的增多和复杂度的提高，对比问题一中的拟牛顿法，相对误差也随之增大，但能保证  $\Delta e \leq 10\%$ ，说明模型具有**较强的鲁棒性**。

# 十一、模型评价

## 11.1 模型的优点

1. 在问题一中利用拟牛顿法建立精确的有效遮蔽判定模型，用于检验简化判定模型误差，兼顾了计算速度和求解结果合理性。
2. 在利用粒子群算法优化时，采用自适应步长，极大地提高收敛速度和计算精度。
3. 在求解问题三、四、五的最优投放策略时，先通过贪心算法，基于第二问模型找到一个局部最优解，即可接受解，从可接受解开始迭代，极大地提高计算了效率

## 11.2 模型的缺点

1. 受限于计算机性能，问题二、三、四、五中均将真目标视为离散的几个代表性点，随着问题的复杂度提高，误差增大。
2. 实际情况中需要考虑，投放干扰弹后不应立即引爆、无人机可以转向、导弹的形状大小等因素。

## 参考文献

- [1] DeepSeek, DeepSeek-R1-0528, 深度求索 (DeepSeek) , 2025-09-04
- [2] 杨唐文, 韩建达, 王红波, 等. 基于空间几何约束的单目视觉物体测距 [C]//2009年中国智能化会议论文集 (第七分册) 南京理工大学学报 (增刊).2009:209-214.DOI:ConferenceArticle/5aa00a4bc095d7222063744d.
- [3] 胡振邦, 夏越. 基于 L-BFGS-B 和 CNN-LSTM 的短期光伏功率预测 [J]. 电气技术与经济,2024,(12):110-114.
- [4] 郑天华, 王佳斌, 蔡宇翔, 等. 应用禁忌粒子群算法的车间调度及其并行化实现 [J]. 华侨大学学报 (自然科学版),2023,44(01):104-110.

## 附录 A 支撑材料的文件列表

| 文件名           | 文件说明                      |
|---------------|---------------------------|
| AI 工具使用详情.pdf | AI 工具名称和版本、使用目的、交互记录、修改情况 |
| problem1.py   | 问题一的 python 代码            |
| problem2.py   | 问题二的 python 代码            |
| problem3.py   | 问题三的 python 代码            |
| problem4.py   | 问题四的 python 代码            |
| problem5.py   | 问题五的 python 代码            |
| limit.py      | 求解遮蔽时间上限的 python 代码       |
| result1.xlsx  | 问题三的输出结果                  |
| result2.xlsx  | 问题四的输出结果                  |
| result3.xlsx  | 问题五的输出结果                  |

## 附录 B 解题代码

### 1. 问题一的 python 代码

```
1  import math
2  import numpy as np
3  from scipy.optimize import minimize, fsolve
4
5
6  g = 9.8
7
8  smoke_sink_speed = 3.0
9  smoke_radius = 10.0
10 smoke_duration = 20.0
11
12 m_speed = 300.0
13 mx_xyz_0=[[0,0,0],[20000, 0, 2000],[19000, 600, 2100],[18000,-600,1900]]#导弹
14
15
16 fyx_xyz_0=[[0,0,0],[17800, 0, 1800],[12000, 1400, 1400],[6000, -3000, 700],[11000, 2000, 1800],[
17                                     13000, -2000, 1300]]
18
19 cyl_center = np.array([0.0, 200.0, 0.0])
20 cyl_radius = 7.0
21 cyl_height = 10.0
22
23 def cylinder_point(theta, h):
24     return np.array(cyl_center+[cyl_radius*np.cos(theta), cyl_radius*np.sin(theta), h])
25
26
27 def distance_point_to_segment(P, A, C):
28     AC = C - A
```



```

29 AP = P - A
30
31 t = np.dot(AP, AC) / (np.dot(AC, AC) + 1e-10)
32
33
34 if 0 <= t <= 1:
35     projection = A + t * AC
36     return np.linalg.norm(P - projection)
37
38 elif t < 0:
39     return np.linalg.norm(P - A)
40 else:
41     return np.linalg.norm(P - C)
42
43 def T_blast(m_number, fy_number, fy_direction_angle, fy_speed, t_drop, t_blast):
44     theta = fy_direction_angle/180*math.pi
45     FY_direction = np.array([np.cos(theta), np.sin(theta), 0])
46     t_det = t_drop + t_blast
47     m_xyz_0_vector = np.array(mx_xyz_0[0])-np.array(mx_xyz_0[m_number])
48     m_unit_dir = m_xyz_0_vector / np.linalg.norm(m_xyz_0_vector)
49     m_vel = m_unit_dir * m_speed
50     t_det = t_drop + t_blast
51     drop_xyz = np.array(fyx_xyz_0[fy_number]) + fy_speed * FY_direction * t_drop
52     blast_xyz = np.array(fyx_xyz_0[fy_number]) + fy_speed * FY_direction * t_det - np.array([0, 0, 0.5
                                                    * g * t_blast ** 2])
53     print(f"FY{fy_number}拦截导弹M{m_number}: 投放坐标为{drop_xyz}, 爆炸坐标为{blast_xyz}")
54
55 def M_point(t):
56     t = np.asarray(t)
57     return np.array(mx_xyz_0[m_number]) + m_vel * t
58
59
60 def O_point(t):
61     t = np.asarray(t)
62     return blast_xyz - np.array([0, 0, 3 * (t - t_det)])
63
64
65 def max_distance(t):
66     t = np.asarray(t).item()
67     Mx_xyz_t = M_point(t)
68     O_xyz_t = O_point(t)
69
70
71 def objective(params):
72     theta, h = params
73     C = cylinder_point(theta, h)
74     return -distance_point_to_segment(O_xyz_t, Mx_xyz_t, C)
75
76
77 initial_guess = [0, 8]
78 bounds = [(0, 2 * np.pi), (0, 10)]
79
80
81 result = minimize(objective, initial_guess, bounds=bounds, method='L-BFGS-B')
82
83 return -result.fun

```

```

84
85 def equation_to_solve(t):
86     if np.isscalar(t):
87         return max_distance(t) - smoke_radius
88     else:
89         return np.array([max_distance(ti) - smoke_radius for ti in t])
90
91 def find_solutions():
92
93     t_values = np.linspace(0, 100, 100)
94     f_values = [equation_to_solve(t) for t in t_values]
95
96
97     intervals = []
98     for i in range(len(t_values) - 1):
99         if f_values[i] * f_values[i + 1] <= 0:
100             intervals.append((t_values[i], t_values[i + 1]))
101
102
103     solutions = []
104     for a, b in intervals:
105         t_sol = fsolve(equation_to_solve, (a + b) / 2)[0]
106
107         if a <= t_sol <= b and abs(equation_to_solve(t_sol)) < 1e-5 :
108
109             M_pos = M_point(t_sol)
110             O_pos = O_point(t_sol)
111             dist_M_to_cyl = np.linalg.norm(M_pos - cyl_center)
112
113             dist_O_to_cyl = np.linalg.norm(O_pos - cyl_center)
114
115
116             smoke_center = np.array([O_pos[0], O_pos[1], O_pos[2] - smoke_sink_speed * (t_sol - t_det)])
117
118             dist_M_to_smoke = np.linalg.norm(M_pos - smoke_center)
119
120             if dist_M_to_smoke <= smoke_radius+1 or dist_M_to_cyl>dist_O_to_cyl-10:
121                 solutions.append(t_sol)
122             solutions = sorted(list(set(solutions)))
123         return solutions
124
125     solutions = find_solutions()
126
127     if len(solutions) >= 2:
128         for i, t in enumerate(solutions[:2]):
129             print(f"t{i + 1} = {t:.6f}")
130             print(f"验证: P(t{i + 1}) = {max_distance(t):.6f}")
131             print(f"FY{fy_number}拦截导弹M{m_number}: 遮蔽时间{solutions[1] - solutions[0]}")
132         else:
133             print("没有找到两个解")
134
135
136 if __name__ == "__main__":
137     T_blast(1,1,180,120,1.5,3.6)
138

```

## 2. 问题二的 python 代码

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 plt.rcParams['font.sans-serif'] = ['SimHei']
4 plt.rcParams['axes.unicode_minus'] = False
5 mx_xyz_0=[[0,0,0],[20000, 0, 2000],[19000, 600, 2100],[18000,-600,1900]]
6 fyx_xyz_0=[[0,0,0],[17800, 0, 1800],[12000, 1400, 1400],[6000, -3000, 700],[11000, 2000, 1800],[
7                                     13000, -2000, 1300]]
8
9 def T_blast_max(m_number,fy_number):
10
11     FY_INIT_POINT = np.array(fyx_xyz_0[fy_number])
12
13
14     M_INIT_POINT = np.array(mx_xyz_0[m_number])
15     M_SPEED = 300.0
16
17
18     cylinder_CENTER = np.array([0.0, 200.0, 0.0])
19     cylinder_RADIUS = 7.0
20     cylinder_HEIGHT = 10.0
21
22
23     SMOKE_EFFECTIVE_RADIUS = 10.0
24     SMOKE_EFFECTIVE_DURATION = 20.0
25     SMOKE_SINK_SPEED = 3.0
26     g = 9.8
27
28
29     FY_SPEED_MIN = 70.0
30     FY_SPEED_MAX = 140.0
31
32
33 def calculate_missile_position(current_time):
34
35     direction = -M_INIT_POINT / np.linalg.norm(M_INIT_POINT)
36     distance_flown = M_SPEED * current_time
37     max_distance = np.linalg.norm(M_INIT_POINT)
38     if distance_flown >= max_distance:
39         return np.array([0.0, 0.0, 0.0])
40     missile_pos = M_INIT_POINT + direction * distance_flown
41     return missile_pos
42
43
44 def calculate_smoke_center(explode_point, explode_time, current_time):
45
46     if current_time < explode_time or current_time > explode_time + SMOKE_EFFECTIVE_DURATION:
47         return np.array([np.inf, np.inf, np.inf])
48     sink_time = current_time - explode_time
49     sink_distance = SMOKE_SINK_SPEED * sink_time
50     smoke_center = explode_point - np.array([0.0, 0.0, sink_distance])
51     return smoke_center
52
53
54 def is_smoke_effective(missile_pos, smoke_center, cylinder_center, cylinder_radius):
55     cylinder_points = [
56         cylinder_center,
57         cylinder_center + np.array([0.0, 0.0, cylinder_HEIGHT]),
58         cylinder_center + np.array([cylinder_radius, 0.0, 0.0]),
59         cylinder_center + np.array([cylinder_radius*np.cos(np.pi/4), cylinder_radius*np.sin(np.pi/4), 0.0]),
60         cylinder_center + np.array([0.0, cylinder_radius, 0.0]),
61         cylinder_center + np.array([cylinder_radius*np.cos(3*np.pi/4), cylinder_radius*np.sin(3*np.pi/4), 0.0])
62     ]
```

```

0]],
55 cylinder_center + np.array([-cylinder_radius, 0.0, 0.0]),
56 cylinder_center + np.array([cylinder_radius*np.cos(5*np.pi/4), cylinder_radius*np.sin(5*np.pi/4), 0.
0]),
57 cylinder_center + np.array([0.0, -cylinder_radius, 0.0]),
58 cylinder_center + np.array([cylinder_radius*np.cos(7*np.pi/4), cylinder_radius*np.sin(7*np.pi/4), 0.
0]),
59 cylinder_center + np.array([cylinder_radius, 0.0, cylinder_HEIGHT]),
60 cylinder_center + np.array([cylinder_radius*np.cos(np.pi/4), cylinder_radius*np.sin(np.pi/4),
cylinder_HEIGHT]),
61 cylinder_center + np.array([0.0, cylinder_radius, cylinder_HEIGHT]),
62 cylinder_center + np.array([cylinder_radius*np.cos(3*np.pi/4), cylinder_radius*np.sin(3*np.pi/4),
cylinder_HEIGHT]),
63 cylinder_center + np.array([-cylinder_radius, 0.0, cylinder_HEIGHT]),
64 cylinder_center + np.array([cylinder_radius*np.cos(5*np.pi/4), cylinder_radius*np.sin(5*np.pi/4),
cylinder_HEIGHT]),
65 cylinder_center + np.array([0.0, -cylinder_radius, cylinder_HEIGHT]),
66 cylinder_center + np.array([cylinder_radius*np.cos(7*np.pi/4), cylinder_radius*np.sin(7*np.pi/4),
cylinder_HEIGHT]),
67 cylinder_center + np.array([cylinder_radius, 0.0, cylinder_HEIGHT/4]),
68 cylinder_center + np.array([0.0, cylinder_radius, cylinder_HEIGHT/4]),
69 cylinder_center + np.array([-cylinder_radius, 0.0, cylinder_HEIGHT/4]),
70 cylinder_center + np.array([0.0, -cylinder_radius, cylinder_HEIGHT/4]),
71 cylinder_center + np.array([cylinder_radius, 0.0, cylinder_HEIGHT*3/4]),
72 cylinder_center + np.array([0.0, cylinder_radius, cylinder_HEIGHT*3/4]),
73 cylinder_center + np.array([-cylinder_radius, 0.0, cylinder_HEIGHT*3/4]),
74 cylinder_center + np.array([0.0, -cylinder_radius, cylinder_HEIGHT*3/4]),
75 cylinder_center + np.array([cylinder_radius, 0.0, cylinder_HEIGHT/2]),
76 cylinder_center + np.array([0.0, cylinder_radius, cylinder_HEIGHT/2]),
77 cylinder_center + np.array([-cylinder_radius, 0.0, cylinder_HEIGHT/2]),
78 cylinder_center + np.array([0.0, -cylinder_radius, cylinder_HEIGHT/2]),
79 cylinder_center + np.array([cylinder_radius, 0.0, cylinder_HEIGHT/8]),
80 cylinder_center + np.array([0.0, cylinder_radius, cylinder_HEIGHT/8]),
81 cylinder_center + np.array([cylinder_radius, 0.0, cylinder_HEIGHT*7/8]),
82 cylinder_center + np.array([0.0, cylinder_radius, cylinder_HEIGHT*7/8]),
83 cylinder_center + np.array([cylinder_radius*np.cos(np.pi/4), cylinder_radius*np.sin(np.pi/4),
cylinder_HEIGHT/4]),
84 cylinder_center + np.array([cylinder_radius*np.cos(np.pi/4), cylinder_radius*np.sin(np.pi/4),
cylinder_HEIGHT/2]),
85 cylinder_center + np.array([cylinder_radius*np.cos(np.pi/4), cylinder_radius*np.sin(np.pi/4),
cylinder_HEIGHT*3/4]),
86 cylinder_center + np.array([cylinder_radius*np.cos(np.pi/4), cylinder_radius*np.sin(np.pi/4),
cylinder_HEIGHT*7/8]),
87 cylinder_center + np.array([cylinder_radius*np.cos(3*np.pi/4), cylinder_radius*np.sin(3*np.pi/4),
cylinder_HEIGHT/4]),
88 cylinder_center + np.array([cylinder_radius*np.cos(3*np.pi/4), cylinder_radius*np.sin(3*np.pi/4),
cylinder_HEIGHT*3/4]),
89 cylinder_center + np.array([cylinder_radius*np.cos(5*np.pi/4), cylinder_radius*np.sin(5*np.pi/4),
cylinder_HEIGHT/4]),
90 cylinder_center + np.array([cylinder_radius*np.cos(5*np.pi/4), cylinder_radius*np.sin(5*np.pi/4),
cylinder_HEIGHT*3/4]),
91 cylinder_center + np.array([cylinder_radius*np.cos(7*np.pi/4), cylinder_radius*np.sin(7*np.pi/4),
cylinder_HEIGHT/4]),
92 cylinder_center + np.array([cylinder_radius*np.cos(7*np.pi/4), cylinder_radius*np.sin(7*np.pi/4),
cylinder_HEIGHT*3/4]),
93 cylinder_center + np.array([cylinder_radius/2, 0.0, cylinder_HEIGHT/4]),

```

```

94 cylinder_center + np.array([0.0, cylinder_radius/2, cylinder_HEIGHT/2]),
95 cylinder_center + np.array([-cylinder_radius/2, 0.0, cylinder_HEIGHT*3/4]),
96 cylinder_center + np.array([0.0, -cylinder_radius/2, cylinder_HEIGHT*7/8]),
97
98 cylinder_center + np.array([cylinder_radius/2*np.cos(np.pi/4), cylinder_radius/2*np.sin(np.pi/4),
99                               cylinder_HEIGHT/2]),
100 cylinder_center + np.array([cylinder_radius/2*np.cos(3*np.pi/4), cylinder_radius/2*np.sin(3*np.pi/4)
101                               , cylinder_HEIGHT/2]),
102 cylinder_center + np.array([cylinder_radius/2*np.cos(5*np.pi/4), cylinder_radius/2*np.sin(5*np.pi/4)
103                               , cylinder_HEIGHT/2]),
104 cylinder_center + np.array([cylinder_radius/2*np.cos(7*np.pi/4), cylinder_radius/2*np.sin(7*np.pi/4)
105                               , cylinder_HEIGHT/2]),
106 cylinder_center + np.array([cylinder_radius/2, 0.0, cylinder_HEIGHT]),
107 cylinder_center + np.array([0.0, cylinder_radius/2, cylinder_HEIGHT]),
108 cylinder_center + np.array([-cylinder_radius/2, 0.0, cylinder_HEIGHT]),
109 cylinder_center + np.array([0.0, -cylinder_radius/2, cylinder_HEIGHT]),
110 cylinder_center + np.array([cylinder_radius/2, 0.0, 0.0]),
111 cylinder_center + np.array([0.0, cylinder_radius/2, 0.0]),
112 cylinder_center + np.array([-cylinder_radius/2, 0.0, 0.0]),
113 cylinder_center + np.array([0.0, -cylinder_radius/2, 0.0]),
114 cylinder_center + np.array([cylinder_radius*0.75, 0.0, cylinder_HEIGHT/2]),
115 cylinder_center + np.array([0.0, cylinder_radius*0.75, cylinder_HEIGHT/2]),
116
117 cylinder_center + np.array([cylinder_radius*0.75*np.cos(np.pi/4), cylinder_radius*0.75*np.sin(np.pi/
118                               4), cylinder_HEIGHT/2]),
119 cylinder_center + np.array([0.0, 0.0, cylinder_HEIGHT/8]),
120 cylinder_center + np.array([0.0, 0.0, cylinder_HEIGHT/4]),
121 cylinder_center + np.array([0.0, 0.0, cylinder_HEIGHT/2]),
122 cylinder_center + np.array([0.0, 0.0, cylinder_HEIGHT*3/4]),
123
124 cylinder_center + np.array([cylinder_radius*np.cos(np.pi/8), cylinder_radius*np.sin(np.pi/8),
125                               cylinder_HEIGHT/4]),
126 cylinder_center + np.array([cylinder_radius*np.cos(3*np.pi/8), cylinder_radius*np.sin(3*np.pi/8),
127                               cylinder_HEIGHT*3/4]),
128
129 for target_point in cylinder_points:
130
131     line_vec = target_point - missile_pos
132     line_len = np.linalg.norm(line_vec)
133     if line_len < 1e-6:
134         continue
135
136     line_unit_vec = line_vec / line_len
137     missile_to_smoke = smoke_center - missile_pos
138     cross_product = np.cross(missile_to_smoke, line_unit_vec)
139     min_distance = np.linalg.norm(cross_product)
140     projection = np.dot(missile_to_smoke, line_unit_vec)
141     if not (0 <= projection <= line_len and min_distance <= SMOKE_EFFECTIVE_RADIUS):
142         return False

```

```

143 return True
144
145
146 def compute_total_cover_time(params):
147
148     theta, v_FY, t_drop, t_explode = params
149
150     flight_dir = np.array([np.cos(theta), np.sin(theta), 0.0])
151     FY_velocity = flight_dir * v_FY
152
153     drop_point = FY_INIT_POINT + FY_velocity * t_drop
154
155     explode_time = t_drop + t_explode
156     vertical_drop = 0.5 * g * (t_explode ** 2)
157     horizontal_movement = flight_dir * v_FY * t_explode
158
159
160     explode_point = drop_point + horizontal_movement - np.array([0.0, 0.0, vertical_drop])
161
162     missile_max_time = np.linalg.norm(M_INIT_POINT) / M_SPEED
163
164     time_step_min = 0.001
165     time_step_max = 0.1
166     total_cover = 0.0
167     is_covering = False
168     cover_start = 0.0
169
170
171     start_time = explode_time
172     end_time = min(explode_time + SMOKE_EFFECTIVE_DURATION, missile_max_time)
173     if start_time >= end_time:
174         return 0.0
175
176
177     initial_distance_to_target = np.linalg.norm(M_INIT_POINT - cylinder_CENTER)
178     t = start_time
179     while t < end_time:
180
181         missile_pos = calculate_missile_position(t)
182         smoke_center = calculate_smoke_center(explode_point, explode_time, t)
183         if np.isinf(smoke_center[0]):
184             current_effective = False
185         else:
186
187             current_effective = is_smoke_effective(
188                 missile_pos, smoke_center, cylinder_CENTER, cylinder_RADIUS
189             )
190
191
192         if current_effective and not is_covering:
193             is_covering = True
194             cover_start = t
195         elif not current_effective and is_covering:
196             is_covering = False
197             total_cover += (t - cover_start)
198

```

```

199
200 distance_to_target = np.linalg.norm(missile_pos - cylinder_CENTER)
201
202 distance_ratio = np.clip(distance_to_target / max(initial_distance_to_target, 1e-6), 0.0, 1.0)
203 adaptive_dt = time_step_min + (time_step_max - time_step_min) * distance_ratio
204
205 if t + adaptive_dt > end_time:
206     adaptive_dt = end_time - t
207     t += adaptive_dt
208
209 if is_covering:
210     total_cover += (end_time - cover_start)
211     return total_cover
212
213
214
215 def pso_optimize(objective_func, dim, bounds, num_particles=50, max_iter=100):
216
217     particles_pos = np.zeros((num_particles, dim))
218     for i in range(dim):
219         particles_pos[:, i] = np.random.uniform(bounds[i][0], bounds[i][1], num_particles)
220
221     particles_vel = np.zeros((num_particles, dim))
222     for i in range(dim):
223         vel_range = 0.5 * (bounds[i][1] - bounds[i][0])
224         particles_vel[:, i] = np.random.uniform(-vel_range, vel_range, num_particles)
225
226
227     pbest_pos = particles_pos.copy()
228     pbest_val = np.array([objective_func(pos) for pos in particles_pos])
229     gbest_idx = np.argmin(pbest_val)
230     gbest_pos = pbest_pos[gbest_idx].copy()
231     gbest_val = pbest_val[gbest_idx]
232     gbest_history = [gbest_val]
233
234
235     c1 = 2.0
236     c2 = 2.0
237     w_max = 0.9
238     w_min = 0.4
239
240
241     for iter in range(max_iter):
242
243         w = w_max - (w_max - w_min) * (iter / max_iter)
244
245         for i in range(num_particles):
246
247             r1 = np.random.random(dim)
248             r2 = np.random.random(dim)
249             particles_vel[i] = (w * particles_vel[i] +
250 c1 * r1 * (pbest_pos[i] - particles_pos[i]) +
251 c2 * r2 * (gbest_pos - particles_pos[i]))
252
253         for d in range(dim):
254             vel_max = 0.5 * (bounds[d][1] - bounds[d][0])

```

```

255 particles_vel[i][d] = np.clip(particles_vel[i][d], -vel_max, vel_max)
256
257
258 particles_pos[i] += particles_vel[i]
259
260 for d in range(dim):
261     particles_pos[i][d] = np.clip(particles_pos[i][d], bounds[d][0], bounds[d][1])
262
263
264 current_val = objective_func(particles_pos[i])
265
266 if current_val < pbest_val[i]:
267     pbest_val[i] = current_val
268     pbest_pos[i] = particles_pos[i].copy()
269
270 if current_val < gbest_val:
271     gbest_val = current_val
272     gbest_pos = particles_pos[i].copy()
273
274
275 gbest_history.append(gbest_val)
276
277 if (iter + 1) % 10 == 0:
278     print(f"PSO迭代 {iter + 1:3d}/{max_iter} | 全局最优目标值: {gbest_val:.4f}")
279
280 return gbest_pos, gbest_val, gbest_history
281
282 def solve_problem_2():
283     print("=" * 60)
284     print("开始求解A题问题2: FY1投放1枚烟幕弹的最优策略")
285     print("=" * 60)
286
287
288     bounds = [
289         (0.0, 2 * np.pi),
290         (FY_SPEED_MIN, FY_SPEED_MAX),
291         (0.2, 60.0),
292         (0.2, 12.0)
293     ]
294     dim = len(bounds)
295
296
297     def pso_objective(params):
298         cover_time = compute_total_cover_time(params)
299         return -cover_time
300
301
302     num_particles = 80
303     max_iter = 160
304
305
306     print("\n启动粒子群优化 (PSO) ...")
307     gbest_pos, gbest_val, gbest_history = pso_optimize(
308         objective_func=pso_objective,
309         dim=dim,
310         bounds=bounds,

```



```

311 num_particles=num_particles,
312 max_iter=max_iter
313 )
314
315
316 theta_opt, v_FY_opt, t_drop_opt, t_explode_opt = gbest_pos
317 max_cover_time = -gbest_val
318
319 flight_dir_opt = np.array([np.cos(theta_opt), np.sin(theta_opt), 0.0])
320 FY_displacement = flight_dir_opt * v_FY_opt * t_drop_opt
321 drop_point_opt = FY_INIT_POINT + FY_displacement
322
323
324 horizontal_movement_opt = flight_dir_opt * v_FY_opt * t_explode_opt
325 vertical_drop_opt = 0.5 * g * (t_explode_opt ** 2)
326 explode_point_opt = drop_point_opt + horizontal_movement_opt - np.array([0.0, 0.0, vertical_drop_opt
327                                     ])
328
329
330 print("\n" + "=" * 60)
331 print("A题问题2 最优投放策略结果")
332 print("=" * 60)
333 print("1. 无人机FY1飞行参数: ")
334 print(f" - 水平飞行方向角: {theta_opt:.4f} 弧度 ({theta_opt * 180 / np.pi:.2f} °)")
335 print(f" - 飞行速度: {v_FY_opt:.2f} m/s (满足70~140m/s约束)")
336 print("2. 烟幕弹投放参数: ")
337 print(f" - 受领任务后投放延迟: {t_drop_opt:.3f} s")
338 print(f" - 投放点坐标 (x, y, z): ({drop_point_opt[0]:.3f}, {drop_point_opt[1]:.2f}, {drop_point_opt
339                                     [2]:.2f}) m")
340
341 print("3. 烟幕弹起爆参数: ")
342 print(f" - 投放后起爆延迟: {t_explode_opt:.3f} s")
343 print(f" - 受领任务后起爆时间: {explode_time_opt:.3f} s")
344 print(
345     f" - 起爆点坐标 (x, y, z): ({explode_point_opt[0]:.3f}, {explode_point_opt[1]:.2f}, {
346                                     explode_point_opt[2]:.2f}) m")
347
348 print(f" - 投放后水平位移: {np.linalg.norm(horizontal_movement_opt):.2f} m")
349 print(f" - 投放后垂直下落: {vertical_drop_opt:.3f} m")
350
351 print("4. 有效遮蔽性能: ")
352 print(f" - 总有效遮蔽时长: {max_cover_time:.3f} s (理论上限20s)")
353 print("=" * 60)
354
355
356 plt.figure(figsize=(10, 6))
357 plt.plot(gbest_history, linewidth=2.0, color='#2E86AB', marker='o', markersize=3, label='PSO收敛曲线
358         ')
359
360 plt.xlabel("PSO迭代次数", fontsize=12)
361 plt.ylabel("目标函数值 (有效遮蔽时长)", fontsize=12)
362 plt.title("粒子群优化收敛曲线", fontsize=14, fontweight='bold')
363 plt.grid(True, alpha=0.3, linestyle='--')
364 plt.axhline(y=-SMOKE_EFFECTIVE_DURATION, color='red', linestyle=':',
365             label=f"理论上限 ({SMOKE_EFFECTIVE_DURATION}s)")
366
367
368 final_iteration = len(gbest_history) - 1
369 final_value = gbest_history[-1]

```

```

363 plt.plot(final_iteration, final_value, marker='*', markersize=12, color='gold',
364 label=f'最终结果({final_value:.3f})', markeredgewidth=0.5)
365
366
367 plt.legend(fontsize=10, loc='upper right')
368 plt.tight_layout()
369 plt.show()
370
371
372 return {
373     "flight_direction_rad": theta_opt,
374     "flight_speed_mps": v_FY_opt,
375     "drop_delay_s": t_drop_opt,
376     "drop_point": drop_point_opt,
377     "explode_delay_s": t_explode_opt,
378     "explode_point": explode_point_opt,
379     "max_cover_time_s": max_cover_time
380 }
381
382 optimal_strategy = solve_problem_2()
383
384 if __name__ == "__main__":
385     T_blast_max(1, 1)

```

### 3. 问题三的 python 代码

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import pandas as pd
4 from pandas import DataFrame
5
6 mx_xyz_0 = [[0, 0, 0], [20000, 0, 2000], [19000, 600, 2100], [18000, -600, 1900]]
7 fyx_xyz_0 = [[0, 0, 0], [17800, 0, 1800], [12000, 1400, 1400], [6000, -3000, 700], [11000, 2000,
8               1800],
9               [13000, -2000, 1300]]
10
11 def T_blast_max_x3(m_number, fy_number):
12     FY_INIT_POINT = np.array(fyx_xyz_0[fy_number])
13     M_INIT_POINT = np.array(mx_xyz_0[m_number])
14     M_SPEED = 300.0
15     cylinder_CENTER = np.array([0.0, 200.0, 0.0])
16     cylinder_RADIUS = 7.0
17     cylinder_HEIGHT = 10.0
18     SMOKE_RADIUS = 10.0
19     SMOKE_DURATION = 20.0
20     SMOKE_SINK_SPEED = 3.0
21
22     g = 9.8
23     FY_SPEED_MIN = 70.0
24     FY_SPEED_MAX = 140.0
25
26     # 中文字体设置
27     plt.rcParams['font.sans-serif'] = ['SimHei']
28     plt.rcParams['axes.unicode_minus'] = False
29

```

```

30 def calculate_missile_position(t):
31
32 dir_vec = -M_INIT_POINT / np.linalg.norm(M_INIT_POINT)
33 flight_dist = M_SPEED * t
34 max_dist = np.linalg.norm(M_INIT_POINT)
35 if flight_dist >= max_dist:
36 return np.array([0.0, 0.0, 0.0])
37 return M_INIT_POINT + dir_vec * flight_dist
38
39 def calculate_smoke_center(explode_point, explode_time, t):
40
41 if t < explode_time or t > explode_time + SMOKE_DURATION:
42 return np.array([np.inf, np.inf, np.inf])
43 sink_dist = SMOKE_SINK_SPEED * (t - explode_time)
44 return explode_point - np.array([0.0, 0.0, sink_dist])
45
46 def is_smoke_effective(missile_pos, smoke_center):
47
48
49 dist_M_to_cyl = np.linalg.norm(missile_pos - cylinder_CENTER)
50 dist_O_to_cyl = np.linalg.norm(smoke_center - cylinder_CENTER)
51 if dist_M_to_cyl <= dist_O_to_cyl:
52 return False
53 target_points = [
54 cylinder_CENTER,
55 cylinder_CENTER + [0, 0, cylinder_HEIGHT],
56 *[cylinder_CENTER + [cylinder_RADIUS * np.cos(a), cylinder_RADIUS * np.sin(a), 0]
57 for a in np.linspace(0, 2 * np.pi, 8, endpoint=False)],
58 *[cylinder_CENTER + [cylinder_RADIUS * np.cos(a), cylinder_RADIUS * np.sin(a), cylinder_HEIGHT]
59 for a in np.linspace(0, 2 * np.pi, 8, endpoint=False)],
60 *[cylinder_CENTER + [cylinder_RADIUS * dx, cylinder_RADIUS * dy, cylinder_HEIGHT / 2]
61 for dx, dy in [(1, 0), (0, 1), (-1, 0), (0, -1)]]
62 ]
63 for tp in target_points:
64 line_vec = tp - missile_pos
65 line_len = np.linalg.norm(line_vec)
66 if line_len < 1e-6:
67 continue
68 line_unit = line_vec / line_len
69 missile_to_smoke = smoke_center - missile_pos
70 min_dist = np.linalg.norm(np.cross(missile_to_smoke, line_unit))
71 projection = np.dot(missile_to_smoke, line_unit)
72 if not (0 <= projection <= line_len and min_dist <= SMOKE_RADIUS):
73 return False
74 return True
75
76 def get_smoke_cover_intervals(theta, v, t_drop, t_explode):
77
78 flight_dir = np.array([np.cos(theta), np.sin(theta), 0.0])
79 drop_point = FY_INIT_POINT + flight_dir * v * t_drop
80 explode_time = t_drop + t_explode
81 vertical_drop = 0.5 * g * t_explode ** 2
82 explode_point = drop_point + flight_dir * v * t_explode - np.array([0, 0, vertical_drop])
83
84
85 missile_max_time = np.linalg.norm(M_INIT_POINT) / M_SPEED

```

```

86 smoke_start = explode_time
87 smoke_end = min(explode_time + SMOKE_DURATION, missile_max_time)
88 if smoke_start >= smoke_end or explode_point[2] < 0:
89     return []
90
91
92 intervals = []
93 is_covering = False
94 cover_start = 0.0
95 initial_dist = np.linalg.norm(M_INIT_POINT - cylinder_CENTER)
96 t = smoke_start
97
98 while t < smoke_end:
99     missile_pos = calculate_missile_position(t)
100     smoke_center = calculate_smoke_center(explode_point, explode_time, t)
101     if np.isinf(smoke_center[0]):
102         current_eff = False
103     else:
104         current_eff = is_smoke_effective(missile_pos, smoke_center)
105
106
107 if current_eff and not is_covering:
108     is_covering = True
109     cover_start = t
110 elif not current_eff and is_covering:
111     is_covering = False
112     intervals.append((cover_start, t))
113
114 curr_dist = np.linalg.norm(missile_pos - cylinder_CENTER)
115 dist_ratio = np.clip(curr_dist / max(initial_dist, 1e-6), 0, 1)
116 dt = 0.001 + (0.5 - 0.001) * dist_ratio
117 if t + dt > smoke_end:
118     dt = smoke_end - t
119 t += dt
120
121 if is_covering:
122     intervals.append((cover_start, smoke_end))
123
124 if not intervals:
125     return []
126 intervals.sort()
127 merged = [intervals[0]]
128 for curr in intervals[1:]:
129     last = merged[-1]
130     if curr[0] <= last[1]:
131         merged[-1] = (last[0], max(last[1], curr[1]))
132     else:
133         merged.append(curr)
134 return merged
135
136 def compute_total_cover_time_multi(params):
137     theta, v = params[0], params[1]
138     t_drops = [params[2], params[4], params[6]]
139     t_explodes = [params[3], params[5], params[7]]
140     if t_drops[1] < t_drops[0] + 1 or t_drops[2] < t_drops[1] + 1:
141         return 0.0

```

```

142 all_intervals = []
143 for t_drop, t_explode in zip(t_drops, t_explodes):
144     intervals = get_smoke_cover_intervals(theta, v, t_drop, t_explode)
145     all_intervals.extend(intervals)
146 if not all_intervals:
147     return 0.0
148 all_intervals.sort()
149 merged = [all_intervals[0]]
150 for curr in all_intervals[1:]:
151     last = merged[-1]
152     if curr[0] <= last[1]:
153         merged[-1] = (last[0], max(last[1], curr[1]))
154     else:
155         merged.append(curr)
156 return sum(end - start for start, end in merged)
157
158
159 def pso_optimize(objective_func, dim, bounds, num_particles=80, max_iter=150):
160
161     particles_pos = np.zeros((num_particles, dim))
162     for i in range(dim):
163         particles_pos[:, i] = np.random.uniform(bounds[i][0], bounds[i][1], num_particles)
164
165     particles_vel = np.zeros((num_particles, dim))
166     for i in range(dim):
167         vel_range = 0.5 * (bounds[i][1] - bounds[i][0])
168         particles_vel[:, i] = np.random.uniform(-vel_range, vel_range, num_particles)
169
170     pbest_pos = particles_pos.copy()
171     pbest_val = np.array([objective_func(pos) for pos in particles_pos])
172     gbest_idx = np.argmin(pbest_val)
173     gbest_pos = pbest_pos[gbest_idx].copy()
174     gbest_val = pbest_val[gbest_idx]
175     gbest_history = [gbest_val]
176
177     c1, c2 = 2.0, 2.0
178     w_max, w_min = 0.9, 0.4
179
180     tabu_list = []
181     tabu_tenure = 10
182     max_local_search = 5
183
184
185     for iter in range(max_iter):
186         w = w_max - (w_max - w_min) * (iter / max_iter)
187
188         for i in range(num_particles):
189
190             r1, r2 = np.random.random(dim), np.random.random(dim)
191             particles_vel[i] = (w * particles_vel[i] +
192                                c1 * r1 * (pbest_pos[i] - particles_pos[i]) +
193                                c2 * r2 * (gbest_pos - particles_pos[i]))
194
195
196         for d in range(dim):
197             vel_max = 0.5 * (bounds[d][1] - bounds[d][0])

```

```

198 particles_vel[i][d] = np.clip(particles_vel[i][d], -vel_max, vel_max)
199
200
201 particles_pos[i] += particles_vel[i]
202
203
204 for d in range(dim):
205     particles_pos[i][d] = np.clip(particles_pos[i][d], bounds[d][0], bounds[d][1])
206
207     t_drop1 = particles_pos[i][2]
208     t_drop2 = max(particles_pos[i][4], t_drop1 + 1.0)
209     t_drop3 = max(particles_pos[i][6], t_drop2 + 1.0)
210     particles_pos[i][4] = t_drop2
211     particles_pos[i][6] = t_drop3
212
213
214     curr_val = objective_func(particles_pos[i])
215     if curr_val < pbest_val[i]:
216         pbest_val[i] = curr_val
217         pbest_pos[i] = particles_pos[i].copy()
218     if curr_val < gbest_val:
219         gbest_val = curr_val
220         gbest_pos = particles_pos[i].copy()
221
222
223     if iter % 10 == 0 and iter > 0:
224
225         local_best_pos = gbest_pos.copy()
226         local_best_val = gbest_val
227
228
229         for local_iter in range(max_local_search):
230
231             neighbor_solutions = []
232             for _ in range(10):
233                 neighbor_pos = local_best_pos.copy()
234
235                 for d in range(dim):
236                     if np.random.random() < 0.3:
237                         perturbation = (bounds[d][1] - bounds[d][0]) * 0.1 * (np.random.random() - 0.5)
238                         neighbor_pos[d] = np.clip(neighbor_pos[d] + perturbation, bounds[d][0], bounds[d][1])
239
240
241                 neighbor_pos[4] = max(neighbor_pos[4], neighbor_pos[2] + 1.0)
242                 neighbor_pos[6] = max(neighbor_pos[6], neighbor_pos[4] + 1.0)
243
244
245                 in_tabu = False
246                 for tabu_solution in tabu_list:
247                     if np.linalg.norm(neighbor_pos - tabu_solution) < 1e-6:
248                         in_tabu = True
249                         break
250
251                 if not in_tabu:
252                     neighbor_solutions.append(neighbor_pos)
253

```

```

254
255 if neighbor_solutions:
256     neighbor_vals = [objective_func(pos) for pos in neighbor_solutions]
257     best_neighbor_idx = np.argmin(neighbor_vals)
258     best_neighbor_val = neighbor_vals[best_neighbor_idx]
259     best_neighbor_pos = neighbor_solutions[best_neighbor_idx]
260
261     if best_neighbor_val < local_best_val:
262         local_best_val = best_neighbor_val
263         local_best_pos = best_neighbor_pos.copy()
264
265
266     if local_best_val < gbest_val:
267         gbest_val = local_best_val
268         gbest_pos = local_best_pos.copy()
269
270
271     tabu_list.append(best_neighbor_pos.copy())
272     if len(tabu_list) > tabu_tenure:
273         tabu_list.pop(0)
274
275
276     gbest_history.append(gbest_val)
277     if (iter + 1) % 10 == 0:
278         print(f"PSO迭代 {iter + 1:3d}/{max_iter} | 全局最优目标值: {gbest_val:.4f}")
279
280
281     return gbest_pos, gbest_val, gbest_history
282
283
284 def solve_problem_3():
285     print("=" * 60)
286     print("开始求解A题问题3: FY1投放3枚烟幕弹的最优策略")
287     print("=" * 60)
288
289
290     bounds = [
291         (0.0, 2 * np.pi), # 0: theta
292         (FY_SPEED_MIN, FY_SPEED_MAX), # 1: v
293         (0.1, 40.0), # 2: t_drop1
294         (0.1, 15.0), # 3: t_explode1
295         (0.1, 45.0), # 4: t_drop2
296         (0.1, 15.0), # 5: t_explode2
297         (0.1, 50.0), # 6: t_drop3
298         (0.1, 15.0) # 7: t_explode3
299     ]
300     dim = len(bounds)
301
302
303     def pso_objective(params):
304         return -compute_total_cover_time_multi(params)
305
306     # 3. 执行PSO优化
307     print("\n启动粒子群优化 (PSO) ...")
308     gbest_pos, gbest_val, gbest_history = pso_optimize(
309         objective_func=pso_objective,

```

```

310 dim=dim,
311 bounds=bounds,
312 num_particles=80,
313 max_iter=180
314 )
315
316
317 theta_opt = gbest_pos[0]
318 v_opt = gbest_pos[1]
319 t_drop1_opt = gbest_pos[2]
320 t_explode1_opt = gbest_pos[3]
321 t_drop2_opt = gbest_pos[4]
322 t_explode2_opt = gbest_pos[5]
323 t_drop3_opt = gbest_pos[6]
324 t_explode3_opt = gbest_pos[7]
325 total_cover_opt = -gbest_val
326
327
328 flight_dir_opt = np.array([np.cos(theta_opt), np.sin(theta_opt), 0.0])
329 FY_vel_opt = flight_dir_opt * v_opt
330
331
332 smoke_detail = []
333 missile_max_time = np.linalg.norm(M_INIT_POINT) / M_SPEED
334 for i, (t_drop, t_explode) in enumerate([
335 (t_drop1_opt, t_explode1_opt),
336 (t_drop2_opt, t_explode2_opt),
337 (t_drop3_opt, t_explode3_opt)
338 ]):
339
340 drop_point = FY_INIT_POINT + FY_vel_opt * t_drop
341
342 explode_time = t_drop + t_explode
343 vertical_drop = 0.5 * g * t_explode ** 2
344 explode_point = drop_point + FY_vel_opt * t_explode - np.array([0, 0, vertical_drop])
345
346 intervals = get_smoke_cover_intervals(theta_opt, v_opt, t_drop, t_explode)
347 single_cover = sum(end - start for start, end in intervals)
348
349 smoke_detail.append({
350     "弹编号": i + 1,
351     "投放延迟(s)": round(t_drop, 3),
352     "投放点X(m)": round(drop_point[0], 3),
353     "投放点Y(m)": round(drop_point[1], 3),
354     "投放点Z(m)": round(drop_point[2], 3),
355     "起爆延迟(s)": round(t_explode, 3),
356     "起爆时间(s)": round(explode_time, 3),
357     "起爆点X(m)": round(explode_point[0], 3),
358     "起爆点Y(m)": round(explode_point[1], 3),
359     "起爆点Z(m)": round(explode_point[2], 3),
360     "单枚遮蔽时间(s)": round(single_cover, 3),
361     "有效区间": [(round(s, 3), round(e, 3)) for s, e in intervals]
362 })
363
364
365 print("\n" + "=" * 60)

```



```

366 print("A题问题3 最优投放策略结果")
367 print("=" * 60)
368 print("1. 无人机FY1飞行参数: ")
369 print(f"    - 水平方向角: {theta_opt:.4f} 弧度 ({theta_opt * 180 / np.pi:.2f} °)")
370 print(f"    - 飞行速度: {v_opt:.2f} m/s (满足70~140m/s约束)")
371 print(f"    - 导弹到达假目标时间: {missile_max_time:.3f} s")
372
373 print("\n2. 3枚烟幕弹详细参数: ")
374 for detail in smoke_detail:
375     print(f"\n    【烟幕弹{detail['弹编号']}】")
376     print(f"    - 投放延迟: {detail['投放延迟(s)']} s")
377     print(f"    - 投放点: ({detail['投放点X(m)']}, {detail['投放点Y(m)']}, {detail['投放点Z(m)']}) m")
378     print(f"    - 起爆延迟: {detail['起爆延迟(s)']} s")
379     print(f"    - 起爆时间: {detail['起爆时间(s)']} s")
380     print(f"    - 起爆点: ({detail['起爆点X(m)']}, {detail['起爆点Y(m)']}, {detail['起爆点Z(m)']}) m")
381     print(f"    - 有效区间: {detail['有效区间']}")
382     print(f"    - 单枚遮蔽时间: {detail['单枚遮蔽时间(s)']} s")
383
384 print("\n3. 整体性能: ")
385 print(f"    - 总有效遮蔽时间: {total_cover_opt:.3f} s")
386 print(f"    - 投放间隔验证: ")
387 print(f"        弹1-弹2: {t_drop2_opt - t_drop1_opt:.3f} s (≥1s)")
388 print(f"        弹2-弹3: {t_drop3_opt - t_drop2_opt:.3f} s (≥1s)")
389 print("=" * 60)
390
391
392 plt.figure(figsize=(10, 6))
393 plt.plot(gbest_history, linewidth=2.0, color='#2E86AB', marker='o', markersize=3)
394 plt.xlabel("PSO迭代次数", fontsize=12)
395 plt.ylabel("负总遮蔽时间 (越小越好)", fontsize=12)
396 plt.title("PSO收敛曲线 (问题3: 3枚烟幕弹)", fontsize=14, fontweight='bold')
397 plt.grid(True, alpha=0.3, linestyle='--')
398 plt.axhline(y=-SMOKE_DURATION * 3, color='orange', linestyle=':', label="理论上限 (60s)")
399 plt.axhline(y=-missile_max_time, color='green', linestyle=':', label=f"导弹飞行时间 ({missile_max_time:.1f}s)")
400 plt.legend()
401 plt.tight_layout()
402 plt.show()
403
404 solve_problem_3()
405
406
407 if __name__ == "__main__":
408     T_blast_max_x3(1, 1)

```

#### 4. 问题四的 python 代码

```

1     import numpy as np
2     from pyswarm import pso
3     from tqdm import tqdm
4
5
6
7     def optimiz():
8         np.random.seed(0)
9

```

```

10 lower_bounds = [8, 7.5, -50, 100, 16, 2.5, 54, 130, 0.5, 1.5, 70, -3]
11 upper_bounds = [9.5, 8.5, -30, 120, 23, 5, 75, 140, 25, 10, 140, 3]
12
13
14 pbar = tqdm(total=5, desc="PSO Optimization", unit="iter")
15
16
17 global_best_fitness = float('-inf')
18 current_iteration = [0]
19
20
21 def objective_function_with_progress(x):
22     fitness = -interference_time(x)
23
24
25     nonlocal global_best_fitness
26     if fitness > global_best_fitness:
27         global_best_fitness = fitness
28
29
30     current_iteration[0] += 1
31     if current_iteration[0] % 150 == 0:
32         pbar.update(1)
33
34
35     return -fitness
36
37 try:
38
39     optimal_solution, optimal_fitness_value = pso(objective_function_with_progress, lower_bounds,
40                                                    upper_bounds,
41                                                    swarmsize=150,
42                                                    maxiter=100,
43                                                    minfunc=1e-6,
44                                                    minstep=1e-6,
45                                                    debug=False,
46                                                    phip=0.5)
47
48     finally:
49
50         pbar.close()
51
52         print('Optimal solution:')
53         print(optimal_solution)
54         print('Best fitness value:', -optimal_fitness_value)
55
56
57 def interference_time(x):
58
59     uav1_t1 = x[8]
60     uav1_t2 = x[9]
61     uav1_v_magnitude = x[10]
62     offset = x[11]
63     rotation_matrix1 = np.array([[np.cos(np.deg2rad(offset)), np.sin(np.deg2rad(offset))],
64                                   [-np.sin(np.deg2rad(offset)), np.cos(np.deg2rad(offset))]])

```

```

65
66 uav2_t1 = x[0]
67 uav2_t2 = x[1]
68 uav2_offset = x[2]
69 uav2_v_magnitude = x[3]
70 rotation_matrix2 = np.array([[np.cos(np.deg2rad(uav2_offset)), np.sin(np.deg2rad(uav2_offset))
71                                ],
72                                [-np.sin(np.deg2rad(uav2_offset)), np.cos(np.deg2rad(uav2_offset))]])
73
74 uav3_t1 = x[4]
75 uav3_t2 = x[5]
76 uav3_offset = x[6]
77 uav3_v_magnitude = x[7]
78 rotation_matrix3 = np.array([[np.cos(np.deg2rad(uav3_offset)), np.sin(np.deg2rad(uav3_offset))
79                                ],
80                                [-np.sin(np.deg2rad(uav3_offset)), np.cos(np.deg2rad(uav3_offset))]])
81
82 m1 = np.array([20000, 0, 2000])
83 m2 = np.array([19000, 600, 2100])
84 m3 = np.array([18000, -600, 1900])
85 uav1 = np.array([17800, 0, 1800])
86 uav2 = np.array([12000, 1400, 1400])
87 uav3 = np.array([6000, -3000, 700])
88 uav4 = np.array([11000, 2000, 1800])
89 uav5 = np.array([13000, -2000, 1300])
90 fake_target = np.array([0, 0, 0])
91
92 true_target = np.array([0, 200, 0])
93 r = 7
94 h = 10
95
96 point_cloud = []
97 for i in np.arange(0, h + 1, 2):
98     angles_deg = np.linspace(0, 360, 19)[:18]
99     angles_rad = np.deg2rad(angles_deg)
100    for radius in [3.5, 7]:
101        x_coord = true_target[0] + radius * np.cos(angles_rad)
102        y_coord = true_target[1] + radius * np.sin(angles_rad)
103        z_coord = true_target[2] + np.ones_like(angles_rad) * i
104        points = np.column_stack((x_coord, y_coord, z_coord))
105        point_cloud.append(points)
106        point_cloud.append(np.array([[true_target[0], true_target[1], i]]))
107    point_cloud = np.vstack(point_cloud)
108
109
110 missile_v = 300
111 g = 9.8
112 cloud_v = -3
113 time_step = 0.01
114
115 v_m1 = missile_v * (fake_target - m1) / np.linalg.norm(fake_target - m1)
116 v_m2 = missile_v * (fake_target - m2) / np.linalg.norm(fake_target - m2)
117 v_m3 = missile_v * (fake_target - m3) / np.linalg.norm(fake_target - m3)
118

```

```

119
120 obscuration_radius = 10
121 obscuration_time = 20
122
123
124 smoke_bombs = [
125     {'release_time': uav1_t1, 'fall_time': uav1_t2, 'uav_init_pos': uav1,
126      'uav_v_magnitude': uav1_v_magnitude, 'rotation_matrix': rotation_matrix1},
127     {'release_time': uav2_t1, 'fall_time': uav2_t2, 'uav_init_pos': uav2,
128      'uav_v_magnitude': uav2_v_magnitude, 'rotation_matrix': rotation_matrix2},
129     {'release_time': uav3_t1, 'fall_time': uav3_t2, 'uav_init_pos': uav3,
130      'uav_v_magnitude': uav3_v_magnitude, 'rotation_matrix': rotation_matrix3}
131 ]
132
133 for i in range(len(smoke_bombs)):
134
135     smoke_bombs[i]['interference_end'] = smoke_bombs[i]['release_time'] + smoke_bombs[i][
136     'fall_time'] + obscuration_time
137
138     direction_vector = fake_target[:2] - smoke_bombs[i]['uav_init_pos'][:2]
139     rotated_direction = smoke_bombs[i]['rotation_matrix'] @ direction_vector
140     v = smoke_bombs[i]['uav_v_magnitude'] * rotated_direction / np.linalg.norm(rotated_direction)
141     smoke_bombs[i]['release_point'] = smoke_bombs[i]['uav_init_pos'][:2] + v * smoke_bombs[i]['
142     release_time']
143
144     release_point_3d = np.append(smoke_bombs[i]['release_point'], smoke_bombs[i]['uav_init_pos'][2
145     ])
146     smoke_bombs[i]['detonation_point'] = release_point_3d + np.append(v * smoke_bombs[i]['
147     fall_time'],
148     -0.5 * g * (smoke_bombs[i]['fall_time'] ** 2))
149     smoke_bombs[i]['detonation_time'] = smoke_bombs[i]['release_time'] + smoke_bombs[i]['fall_time
150     ']
151
152     detonation_times = [smoke_bombs[i]['detonation_time'] for i in range(len(smoke_bombs))]
153     interference_ends = [smoke_bombs[i]['interference_end'] for i in range(len(smoke_bombs))]
154     start_time = min(detonation_times)
155     end_time = max(interference_ends)
156
157     results = []
158     t_values = np.arange(start_time, end_time + time_step, time_step)
159
160     for t in t_values:
161         is_obscured = 0
162         m1_t = m1 + t * v_m1
163
164         missile_to_target = true_target - m1_t
165
166         for idx in range(3):
167             smoke = smoke_bombs[idx]
168
169             if smoke['detonation_time'] <= t <= smoke['interference_end']:
170
171                 cloud_center = smoke['detonation_point'] + np.array([0, 0, cloud_v * (t - smoke['

```

```

171         detonation_time']]])
172
173 missile_to_cloud = cloud_center - m1_t
174
175
176 dot_missile_cloud_target = np.dot(missile_to_cloud, missile_to_target)
177
178
179 cloud_to_target = true_target - cloud_center
180 dot_cloud_target_missile = np.dot(cloud_to_target, missile_to_target)
181
182 is_between_missile_and_target = (dot_missile_cloud_target > 0) and (dot_cloud_target_missile >
183                                0)
184
185 if not is_between_missile_and_target:
186     continue
187
188 distance_missile_to_cloud = np.linalg.norm(missile_to_cloud)
189
190 if distance_missile_to_cloud <= obscuration_radius:
191     is_obsured = 1
192     break
193
194 line_to_points = point_cloud - m1_t
195 line_to_cloud = missile_to_cloud
196
197 dot_products = np.dot(line_to_points, line_to_cloud)
198
199
200 norm_points = np.linalg.norm(line_to_points, axis=1)
201 norm_cloud = np.linalg.norm(line_to_cloud)
202
203
204 valid_indices = (norm_points > 1e-10) & (norm_cloud > 1e-10)
205 if not np.any(valid_indices):
206     continue
207
208
209 cosine_angles = np.zeros(len(point_cloud))
210 cosine_angles[valid_indices] = dot_products[valid_indices] / (norm_points[valid_indices] *
211                                                                norm_cloud)
212
213
214 sine_angles = np.sqrt(1 - np.clip(cosine_angles ** 2, 0, 1))
215 distances = norm_cloud * sine_angles
216
217 points_obsured = (distances <= obscuration_radius) & (cosine_angles > 0)
218
219
220 if np.any(points_obsured):
221     is_obsured = 1
222     break
223

```

```

224     results.append(is_obscured)
225
226     fitness = time_step * np.sum(results)
227     return fitness
228
229
230     if __name__ == '__main__':
231         optimiz()
232

```

#### 4. 问题五的 python 代码

```

1     import math
2     import numpy as np
3     import pandas as pd
4     from pyswarm import pso
5     import warnings
6
7     warnings.filterwarnings("ignore")
8     UAV_CONFIG = {
9         "FY1": {"init_pos": (17800, 0, 1800), "max_missiles": 3},
10        "FY2": {"init_pos": (12000, 1400, 1400), "max_missiles": 3},
11        "FY3": {"init_pos": (6000, -3000, 700), "max_missiles": 3},
12        "FY4": {"init_pos": (11000, 2000, 1800), "max_missiles": 3},
13        "FY5": {"init_pos": (13000, -2000, 1300), "max_missiles": 3}
14    }
15    UAV_LIST = list(UAV_CONFIG.keys())
16
17    MISSILE_CONFIG = {
18        "M1": {"init_pos": (20000, 0, 2000), "speed": 300},
19        "M2": {"init_pos": (19000, 600, 2100), "speed": 300},
20        "M3": {"init_pos": (18000, -600, 1900), "speed": 300}
21    }
22    MISSILE_LIST = list(MISSILE_CONFIG.keys())
23
24    SMOKE_PARAMS = {
25        "g": 9.8, # 重力加速度(m/s²)
26        "sink_speed": 3, # 云团匀速下沉速度(m/s)
27        "effective_radius": 10, # 有效遮蔽半径(m)
28        "effective_time": 20, # 起爆后有效时长(s)
29        "min_drop_interval": 1 # 同架无人机投弹最小间隔(s)
30    }
31
32    TARGET_PARAMS = {
33        "type": "particle", # 理想化为质点
34        "center_bottom": (0, 200, 0) # 质点坐标
35    }
36
37    PSO_PARAMS = {
38        "n_particles": 100, # 粒子数
39        "max_iter": 180, # 迭代次数
40        "omega": 0.65, # 惯性权重
41        "c1": 2, # 个体学习因子
42        "c2": 2 # 群体学习因子
43    }
44

```

```

45 TIME_PARAMS = {
46     "time_step": 0.5, # 时间步长
47     "t_max": None # 导弹最晚到达时间
48 }
49
50 def precompute_missile_positions():
51     missile_t_end = {}
52     for m_name in MISSILE_LIST:
53         x0, y0, z0 = MISSILE_CONFIG[m_name]["init_pos"]
54         dist_total = math.sqrt(x0 ** 2 + y0 ** 2 + z0 ** 2)
55         missile_t_end[m_name] = dist_total / MISSILE_CONFIG[m_name]["speed"]
56     TIME_PARAMS["t_max"] = max(missile_t_end.values())
57
58     t_range = np.arange(0, TIME_PARAMS["t_max"], TIME_PARAMS["time_step"])
59     missile_pos_cache = {}
60     for m_name in MISSILE_LIST:
61         x0, y0, z0 = MISSILE_CONFIG[m_name]["init_pos"]
62         speed = MISSILE_CONFIG[m_name]["speed"]
63         dist_total = math.sqrt(x0 ** 2 + y0 ** 2 + z0 ** 2)
64         dx, dy, dz = -x0 / dist_total, -y0 / dist_total, -z0 / dist_total
65
66         t_vec = t_range.reshape(-1, 1)
67         x_vec = x0 + speed * t_vec * dx
68         y_vec = y0 + speed * t_vec * dy
69         z_vec = z0 + speed * t_vec * dz
70
71         mask = t_range >= missile_t_end[m_name]
72         x_vec[mask] = np.nan
73         y_vec[mask] = np.nan
74         z_vec[mask] = np.nan
75
76     missile_pos_cache[m_name] = np.hstack([x_vec, y_vec, z_vec])
77     return missile_pos_cache, t_range
78
79
80 GLOBAL_TARGET_POINT = np.array(TARGET_PARAMS["center_bottom"])
81 GLOBAL_MISSILE_POS_CACHE, GLOBAL_T_RANGE = precompute_missile_positions()
82
83
84 def calc_smoke_drop_point(uav_name, t_drop, theta, v):
85     x0, y0, z0 = UAV_CONFIG[uav_name]["init_pos"]
86     x = x0 + v * t_drop * math.cos(theta)
87     y = y0 + v * t_drop * math.sin(theta)
88     return (x, y, z0)
89
90
91 def calc_smoke_explode_point(drop_point, t_explode):
92     x_drop, y_drop, z_drop = drop_point
93     z_explode = z_drop - 0.5 * SMOKE_PARAMS["g"] * (t_explode ** 2)
94     z_explode = max(z_explode, 0.1)
95     return (x_drop, y_drop, z_explode)
96
97
98 def calc_smoke_center(explode_point, t_total, t):
99     if not (t_total <= t <= t_total + SMOKE_PARAMS["effective_time"]):
100         return None

```

```

101 x_explode, y_explode, z_explode = explode_point
102 delta_t = t - t_total
103 z_smoke = z_explode - SMOKE_PARAMS["sink_speed"] * delta_t
104 return (x_explode, y_explode, z_smoke)
105
106 def is_segment_intersect_sphere(missile_pos, smoke_center):
107     m_pos = np.array(missile_pos)
108     tar_pos = GLOBAL_TARGET_POINT
109     sc_pos = np.array(smoke_center)
110     radius = SMOKE_PARAMS["effective_radius"]
111
112     v = tar_pos - m_pos
113     w = sc_pos - m_pos
114     a = np.dot(v, v)
115
116     if a < 1e-8:
117         return np.linalg.norm(w) <= radius + 1e-8
118
119     t = max(0, min(1, np.dot(w, v) / a))
120     closest_point = m_pos + t * v
121     dist = np.linalg.norm(closest_point - sc_pos)
122     return dist <= radius + 1e-8
123
124
125 def is_missile_shielded(missile_name, smoke_info_list, t, missile_pos):
126     if np.any(np.isnan(missile_pos)):
127         return True
128     for (drop_point, t_explode, t_drop) in smoke_info_list:
129         t_total = t_drop + t_explode
130         if not (t_total <= t <= t_total + SMOKE_PARAMS["effective_time"]):
131             continue
132         explode_point = calc_smoke_explode_point(drop_point, t_explode)
133         smoke_center = calc_smoke_center(explode_point, t_total, t)
134         if smoke_center is None:
135             continue
136         if is_segment_intersect_sphere(missile_pos, smoke_center):
137             return True
138     return False
139
140 def calculate_total_shield_time(decision_vars):
141     smoke_info_list = []
142     var_idx = 0
143
144     for uav_name in UAV_LIST:
145         theta = decision_vars[var_idx]
146         v = max(70, min(140, decision_vars[var_idx + 1]))
147         var_idx += 2
148
149         t_drop_list = [max(0, decision_vars[var_idx + i]) for i in range(3)]
150         t_explode_list = [max(0.1, decision_vars[var_idx + 3 + i]) for i in range(3)]
151         var_idx += 6
152
153         for i in range(1, 3):
154             if t_drop_list[i] < t_drop_list[i - 1] + SMOKE_PARAMS["min_drop_interval"]:
155                 t_drop_list[i] = t_drop_list[i - 1] + SMOKE_PARAMS["min_drop_interval"]
156

```



```

157     for i in range(3):
158         drop_point = calc_smoke_drop_point(uav_name, t_drop_list[i], theta, v)
159         smoke_info_list.append((drop_point, t_explode_list[i], t_drop_list[i]))
160
161     # 总时长 = M1被遮蔽时间 + M2被遮蔽时间 + M3被遮蔽时间
162     total_shield_time = 0
163     for t_idx, t in enumerate(GLOBAL_T_RANGE):
164         m1_pos = GLOBAL_MISSILE_POS_CACHE["M1"][t_idx]
165         m2_pos = GLOBAL_MISSILE_POS_CACHE["M2"][t_idx]
166         m3_pos = GLOBAL_MISSILE_POS_CACHE["M3"][t_idx]
167
168         if is_missile_shielded("M1", smoke_info_list, t, m1_pos):
169             total_shield_time += TIME_PARAMS["time_step"]
170         if is_missile_shielded("M2", smoke_info_list, t, m2_pos):
171             total_shield_time += TIME_PARAMS["time_step"]
172         if is_missile_shielded("M3", smoke_info_list, t, m3_pos):
173             total_shield_time += TIME_PARAMS["time_step"]
174
175     return total_shield_time
176
177
178     def get_pso_bounds():
179         lower = []
180         upper = []
181         for _ in UAV_LIST:
182             lower.append(0)
183             upper.append(2 * math.pi)
184             lower.append(70)
185             upper.append(140)
186             lower.extend([0, 0, 0])
187             upper.extend([50, 50, 50])
188             lower.extend([0.1, 0.1, 0.1])
189             upper.extend([10, 10, 10])
190         return np.array(lower), np.array(upper)
191
192
193     def init_pso_particles(n_particles, dim):
194         particles = np.random.uniform(*get_pso_bounds(), (n_particles, dim))
195         for i in range(n_particles):
196             var_idx = 0
197             for uav_name in UAV_LIST:
198                 uav_x0, uav_y0, _ = UAV_CONFIG[uav_name]["init_pos"]
199                 target_theta = math.atan2(-uav_y0, -uav_x0)
200                 particles[i, var_idx] = np.random.uniform(target_theta - math.pi / 4, target_theta + math.pi / 4
201
202
203
204
205
206
207
208
209
210
211         def pso_optimize():

```

```

212 lb, ub = get_pso_bounds()
213 init_particles = init_pso_particles(PSO_PARAMS["n_particles"], len(lb))
214
215 def objective_func(decision_vars):
216     return -calculate_total_shield_time(decision_vars) # PSO最小化→最大化总时长
217
218 print(f"PSO优化启动（总遮蔽时长=三枚导弹各自被遮蔽时间之和）：")
219 print(f"粒子数={PSO_PARAMS['n_particles']} | 迭代次数={PSO_PARAMS['max_iter']}")
220 best_vars, best_obj = pso(
221     func=objective_func,
222     lb=lb,
223     ub=ub,
224     swarmsize=PSO_PARAMS["n_particles"],
225     maxiter=PSO_PARAMS["max_iter"],
226     omega=PSO_PARAMS["omega"],
227     phip=PSO_PARAMS["c1"],
228     phig=PSO_PARAMS["c2"],
229     minstep=1e-6,
230     minfunc=1e-6
231 )
232 max_total_shield_time = -best_obj
233 print(f"优化完成！总遮蔽时长（三枚之和）：{max_total_shield_time:.1f}s")
234 return best_vars, max_total_shield_time
235
236 def parse_best_strategy(best_vars):
237     strategy = []
238     var_idx = 0
239     for uav_name in UAV_LIST:
240         # 解析无人机飞行参数
241         theta = best_vars[var_idx]
242         v = max(70, min(140, best_vars[var_idx + 1]))
243         theta_deg = round(math.degrees(theta), 1)
244         var_idx += 2
245
246         # 解析3枚烟幕弹参数
247         t_drop_list = [max(0, round(best_vars[var_idx + i], 1)) for i in range(3)]
248         t_explode_list = [max(0.1, round(best_vars[var_idx + 3 + i], 1)) for i in range(3)]
249         var_idx += 6
250
251         # 强制投弹间隔约束
252         for i in range(1, 3):
253             if t_drop_list[i] < t_drop_list[i - 1] + 1:
254                 t_drop_list[i] = t_drop_list[i - 1] + 1
255
256         # 遍历3枚烟幕弹
257         for i in range(3):
258             t_d = t_drop_list[i]
259             t_e = t_explode_list[i]
260             # 基础烟幕信息
261             drop_point = calc_smoke_drop_point(uav_name, t_d, theta, v)
262             drop_point_str = f"({drop_point[0]:.1f}, {drop_point[1]:.1f}, {drop_point[2]:.1f})"
263             explode_point = calc_smoke_explode_point(drop_point, t_e)
264             explode_point_str = f"({explode_point[0]:.1f}, {explode_point[1]:.1f}, {explode_point[2]:.1f})"
265             t_total = t_d + t_e
266             smoke_self_interval = f"[{t_total:.1f}, {t_total + 20:.1f}]" # 烟幕自身有效区间
267

```

```

268 # 组装策略字典
269 strategy.append({
270     "无人机编号": uav_name,
271     "飞行方向 $\theta(^{\circ})$ ": theta_deg,
272     "飞行速度 $v(m/s)$ ": round(v, 1),
273     "烟幕弹序号": i + 1,
274     "投放延迟 $t_{drop}(s)$ ": t_d,
275     "投放点坐标(x,y,z)": drop_point_str,
276     "起爆延迟 $t_{explode}(s)$ ": t_e,
277     "起爆点坐标(x,y,z)": explode_point_str,
278     "烟幕自身有效区间(s)": smoke_self_interval
279 })
280 return strategy
281
282
283 def verify_strategy(strategy):
284     print("\n" + "=" * 50)
285     print("约束验证结果（总遮蔽时长=三枚导弹各自被遮蔽时间之和）")
286     print("=" * 50)
287     valid = True
288     # 验证速度约束
289     for item in strategy:
290         v = item["飞行速度 $v(m/s)$ "]
291         if not (70 <= v <= 140):
292             print(f"{item['无人机编号']}速度{v}m/s, 超出[70,140]范围")
293             valid = False
294     # 验证投弹间隔约束
295     uav_drop_times = {uav: [] for uav in UAV_LIST}
296     for item in strategy:
297         uav = item["无人机编号"]
298         uav_drop_times[uav].append(item["投放延迟 $t_{drop}(s)$ "])
299     for uav, t_list in uav_drop_times.items():
300         t_list.sort()
301         for i in range(1, 3):
302             if t_list[i] - t_list[i - 1] < 1 - 1e-8:
303                 print(f"{uav}第{i + 1}枚与第{i}枚弹间隔{t_list[i] - t_list[i - 1]:.1f}s, 小于1s")
304                 valid = False
305         if valid:
306             print("所有约束均满足要求")
307         else:
308             print("部分约束未满足, 建议重新优化")
309     print("=" * 50)
310
311
312 def save_strategy_to_excel(strategy, max_total_shield_time, filename="result_sum3.xlsx"):
313     """保存策略到Excel"""
314     # 定义列顺序
315     columns_order = [
316         "无人机编号", "飞行方向 $\theta(^{\circ})$ ", "飞行速度 $v(m/s)$ ", "烟幕弹序号",
317         "投放延迟 $t_{drop}(s)$ ", "投放点坐标(x,y,z)", "起爆延迟 $t_{explode}(s)$ ", "起爆点坐标(x,y,z)",
318         "烟幕自身有效区间(s)"
319     ]
320     df = pd.DataFrame(strategy, columns=columns_order)
321
322     # 新增合计行（标注说明信息）
323     total_row = {col: "-" for col in df.columns}

```

```

324 total_row["无人机编号"] = "策略合计"
325 total_row["飞行方向 $\theta(^{\circ})$ "] = f"总遮蔽时长（三枚之和）：{max_total_shield_time:.1f}s | 质点坐标：(
                                0,200,0)"
326 total_row["烟幕自身有效区间(s)"] = "烟幕起爆后20s（理论有效）"
327
328 df = pd.concat([df, pd.DataFrame([total_row])], ignore_index=True)
329
330 # 保存Excel并调整列宽
331 with pd.ExcelWriter(filename, engine="openpyxl") as writer:
332     df.to_excel(writer, sheet_name="无人机干扰策略", index=False)
333     worksheet = writer.sheets["无人机干扰策略"]
334     # 调整列宽
335     worksheet.column_dimensions["I"].width = 22 # 烟幕自身有效区间
336
337 def main():
338     print("=" * 60)
339     print("求解：5架无人机干扰3枚导弹（总遮蔽时长=三枚各自被遮蔽时间之和）")
340     print("=" * 60)
341     print(f"真目标假设：{TARGET_PARAMS['type']}，坐标={TARGET_PARAMS['center_bottom']}")
342     print("=" * 60)
343
344     best_vars, max_total_shield_time = pso_optimize()
345     print("\n2. 解析最优投放策略...")
346     strategy = parse_best_strategy(best_vars)
347     verify_strategy(strategy)
348     print("\n3. 保存策略到Excel...")
349     save_strategy_to_excel(strategy, max_total_shield_time)
350
351
352 if __name__ == "__main__":
353     main()
354

```